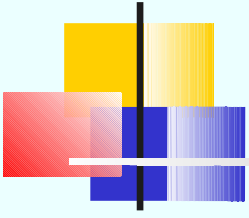


Programming with Android

Android basic

Dr. Nguyen Manh Hung

Posts & Telecommunications Institute of Technology (PTIT)



Content

- Introduction
- Installation and setting
- Emulator
- First app
- Resources
- Activity/service/fragment
- Content provider/SQLite
- Intent & filter
- UI control/event handling
- Style & theme

References



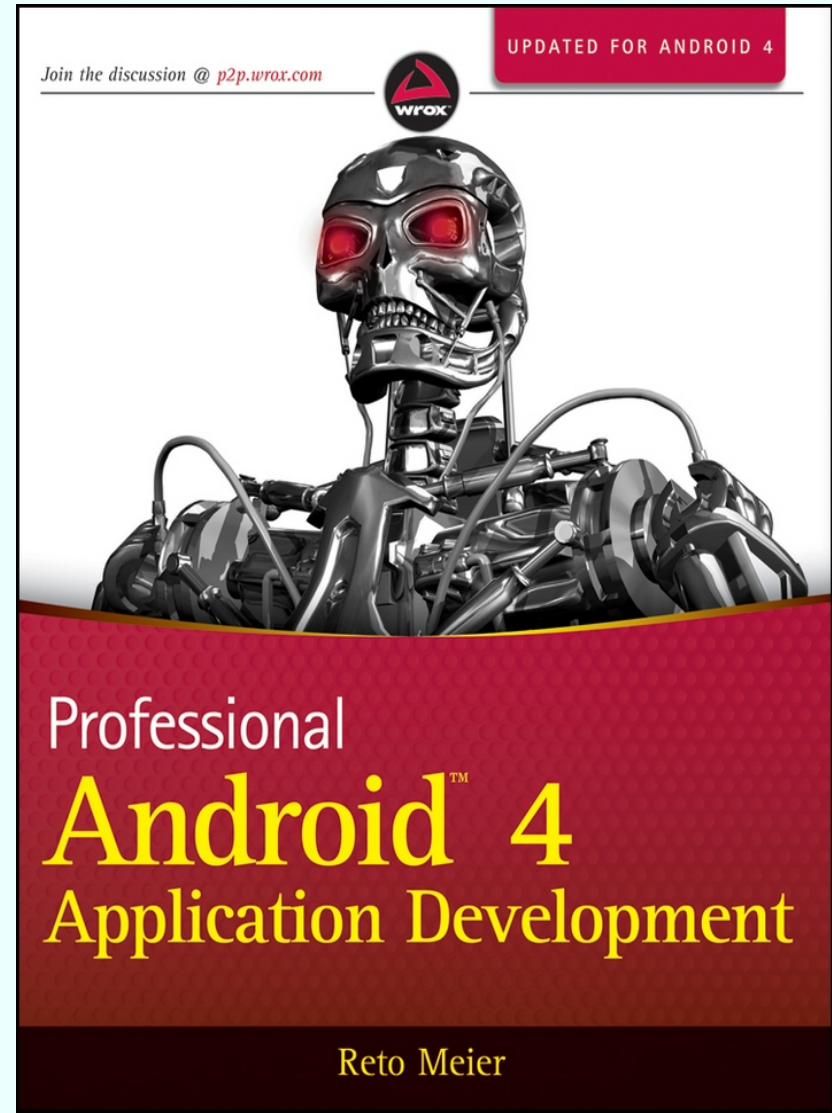
www.tutorialspoint.com

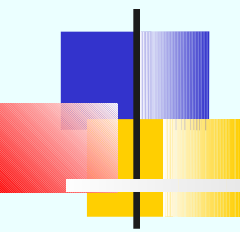


<https://www.facebook.com/tutorialspointindia>

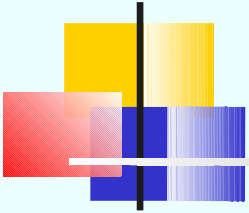


<https://twitter.com/tutorialspoint>





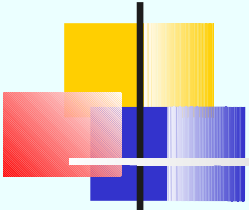
Introduction



History

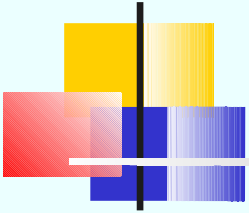
Android:

- Android is an open source and Linux-based Operating System for mobile devices
- The first beta version of the Android Software Development Kit (SDK) was released by Google in 2007, whereas the first commercial version, Android 1.0 was released in September 2008.
- Until 2017, the latest version of Android is 7.x



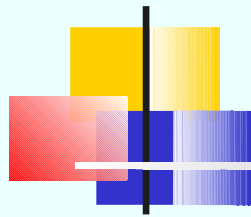
Features (1)

Feature	Description
UI	Android OS basic screen provides a beautiful and intuitive user interface.
Connectivity	GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth, Wi-Fi, LTE, NFC and WiMAX.
Storage	SQLite, a lightweight relational database, is used for data storage purposes.
Media support	H.263, H.264, MPEG-4 SP, AMR, AMR-WB, AAC, HE-AAC, AAC 5.1, MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP
Messaging	SMS and MMS
Web browser	Based on the open-source WebKit layout engine, coupled with Chrome's V8 JavaScript engine supporting HTML5 and CSS3.
Multi-touch	Android has native support for multi-touch which was initially made available in handsets such as the HTC Hero.



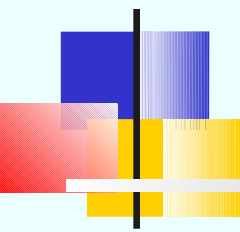
Features (2)

Feature	Description
Multi-tasking	User can jump from one task to another and same time various application can run simultaneously.
Resizable widgets	Widgets are resizable, so users can expand them to show more content or shrink them to save space
Multi-Language	Support single direction and bi-directional text.
GCM	Google Cloud Messaging (GCM) is a service that let developers send short message data to their users on Android devices, without needing a proprietary sync solution.
Wi-Fi Direct	technology that let apps discover and pair directly, over a high-bandwidth peer-to-peer connection.
Android Beam	A popular NFC-based technology that let users instantly share, just by touching two NFC-enabled phones together.

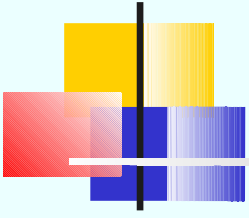


Applications

- Java language
- Android Software Development Kit (SDK)
- Google play
- Amazon Appstore



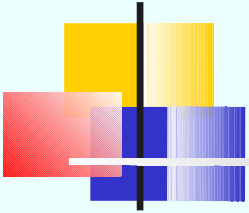
Environment setup



Requirements (1)

OS:

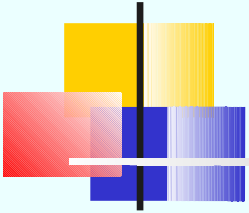
- Microsoft Windows XP or later version.
- Mac OS X 10.5.8 or later version with Intel chip.
- Linux including GNU C Library 2.7 or later.



Requirements (2)

Software and lib/tools:

- Java JDK5 or later
- Android SDK
- Eclipse IDE for Java Developers
- Android Development Tools (ADT) Eclipse Plugin
- Emulators: Genymotion



Setup (1)

JDK:

- Install JDK in a directory
- Setup the environment variables

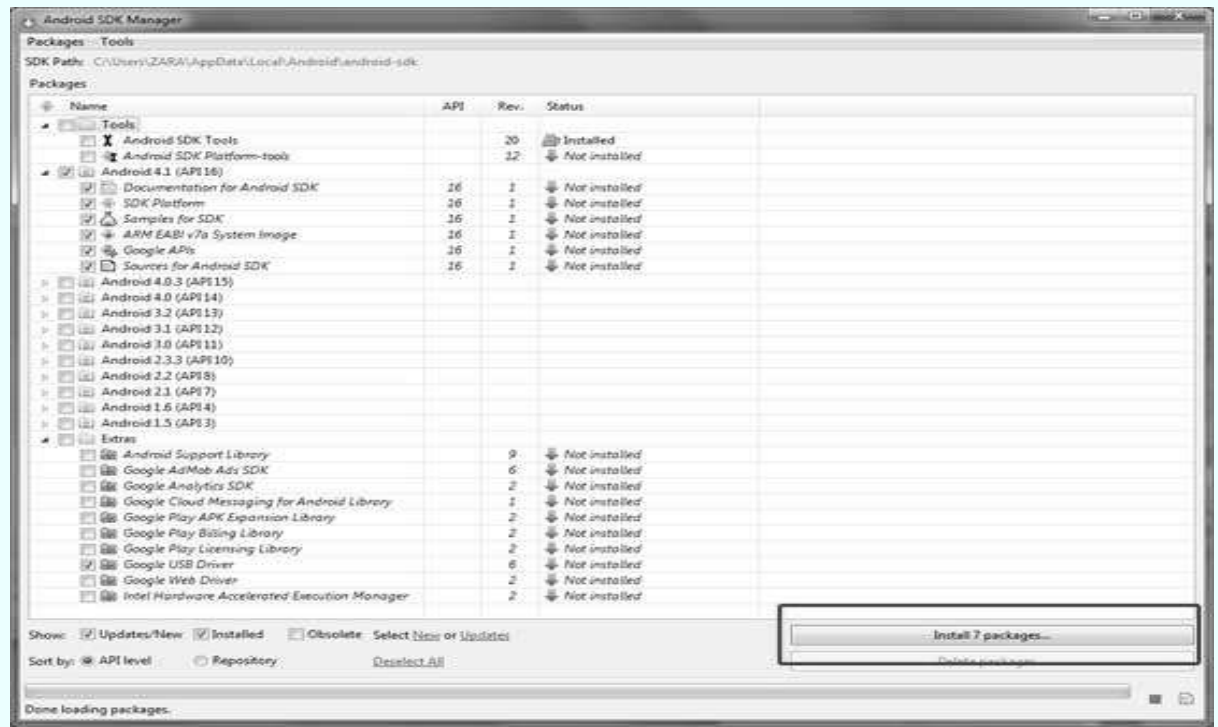
`JAVA_HOME = path to the JDK home directory`

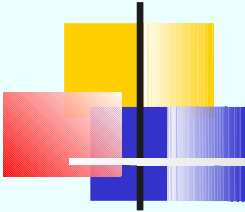
`PATH = %PATH%; JAVA_HOME/bin;`

Setup (2)

Android SDK:

- Install the downloaded android SDK
- All Programs → Android SDK Tools → SDK Manager

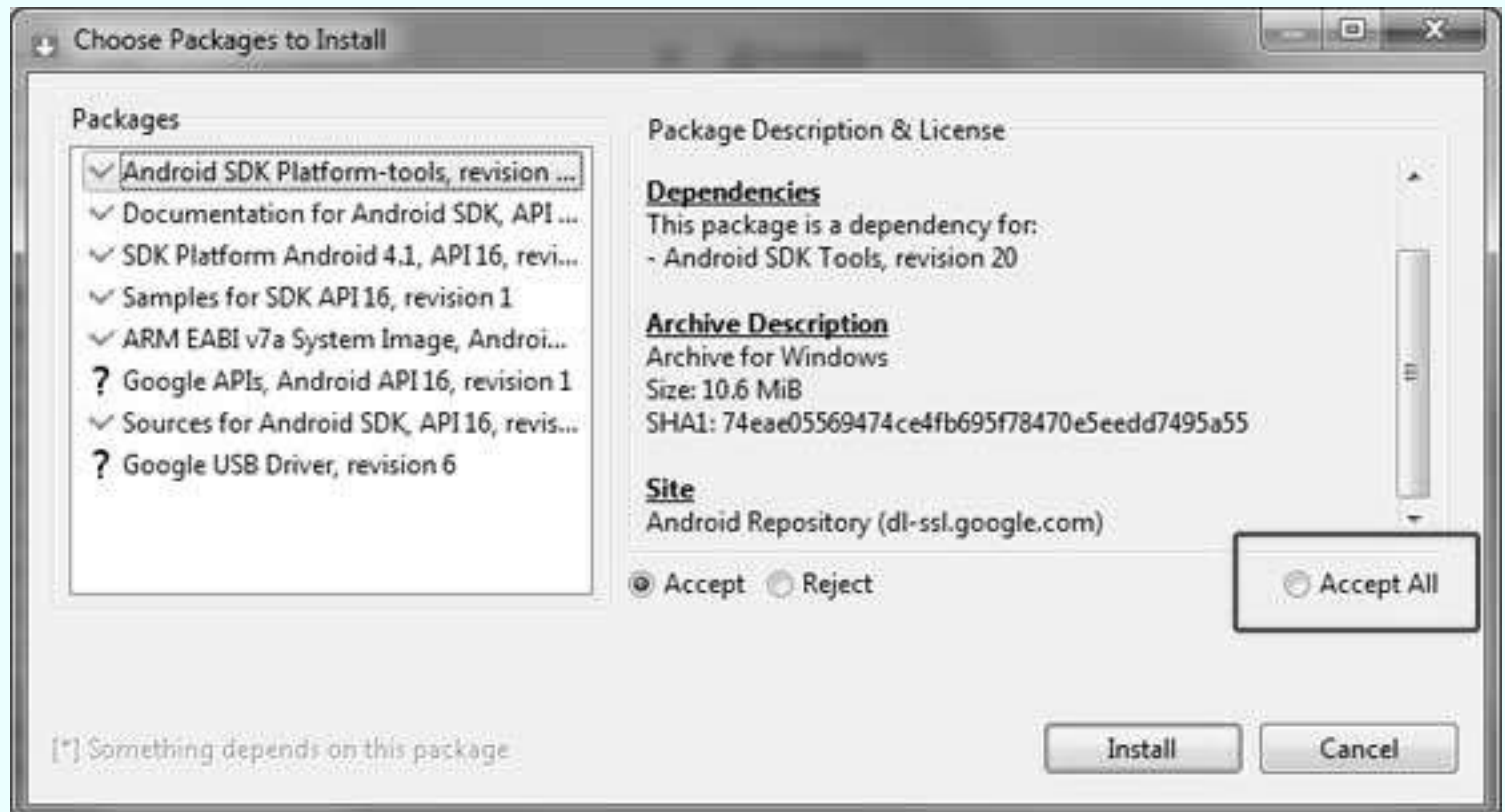


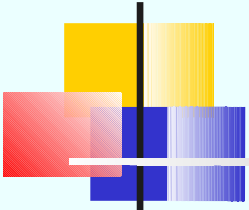


Setup (3)

Android SDK:

- Click install the update packages

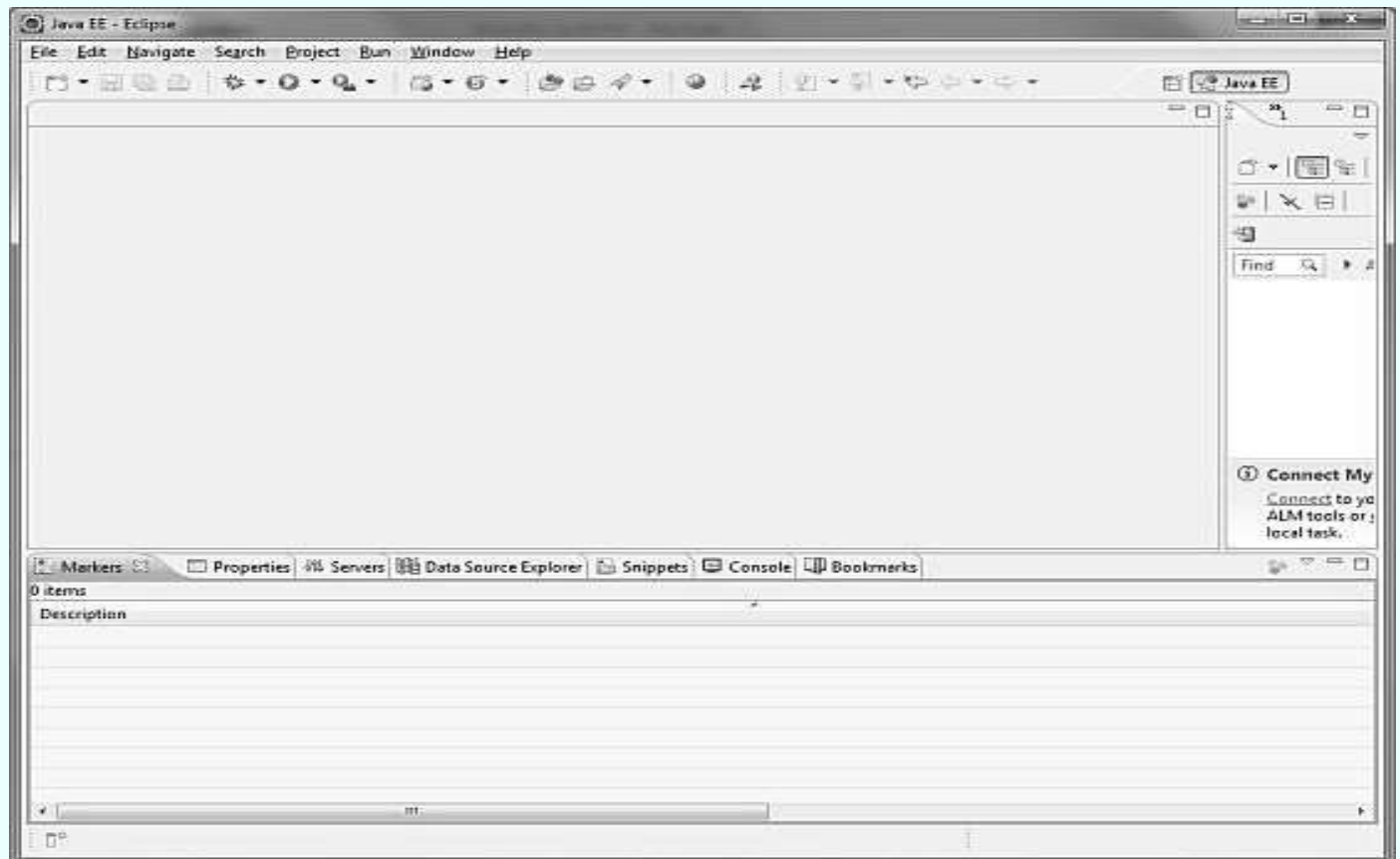




Setup (4)

Eclipse IDE:

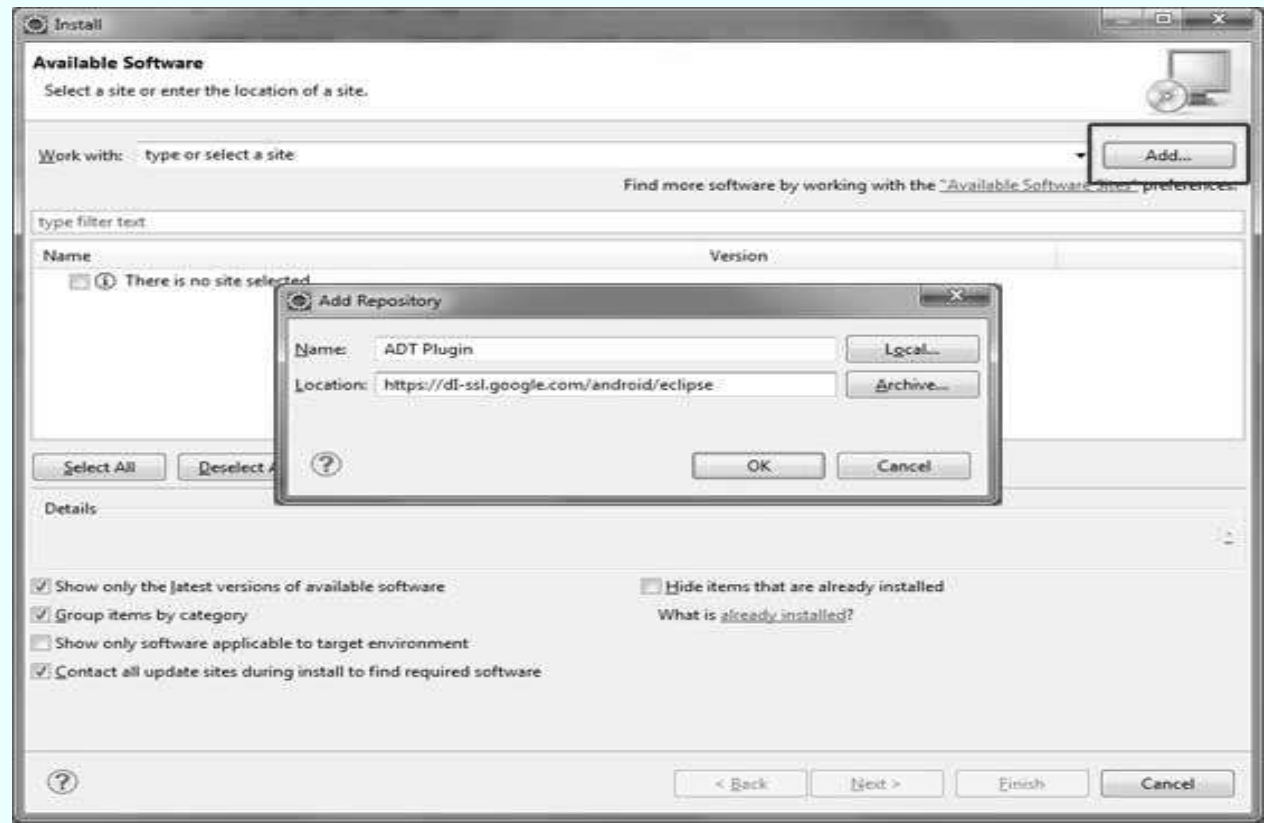
- Download and run the eclipse.exe

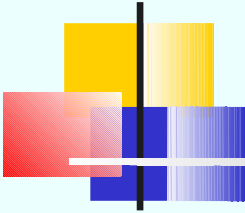


Setup (5)

ADT plugin:

- Run eclipse: Help → Software Updates → Install New Software

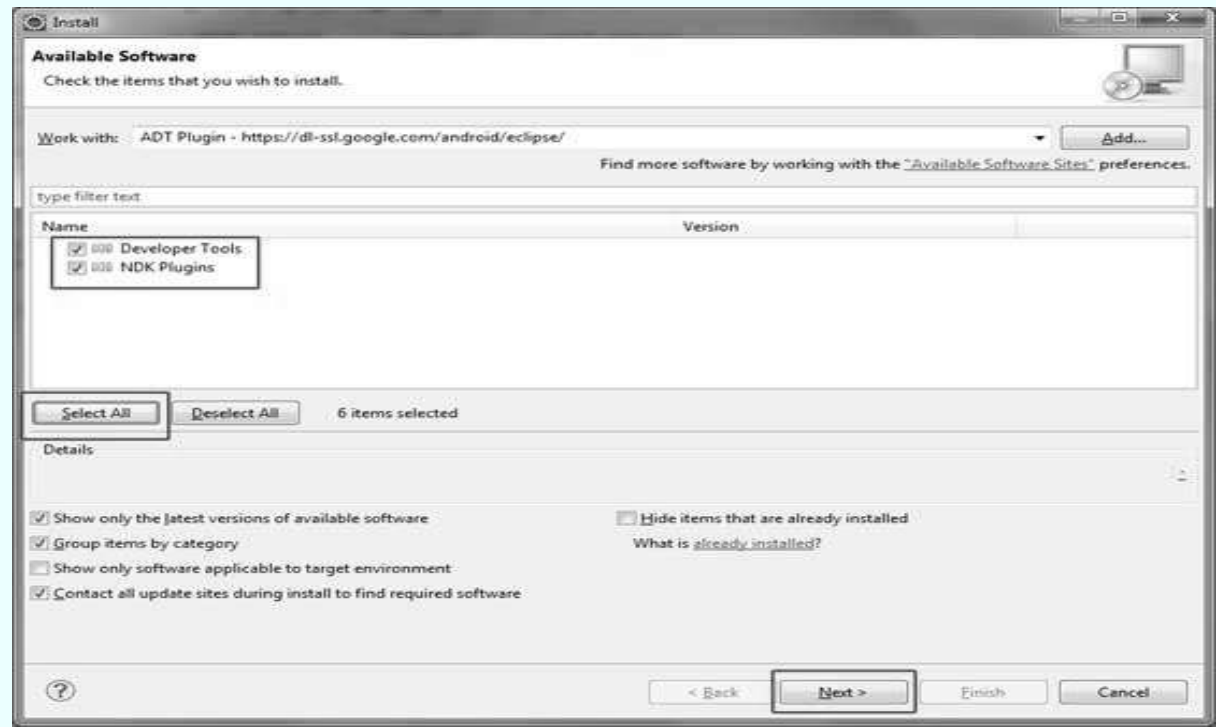




Setup (6)

ADT plugin:

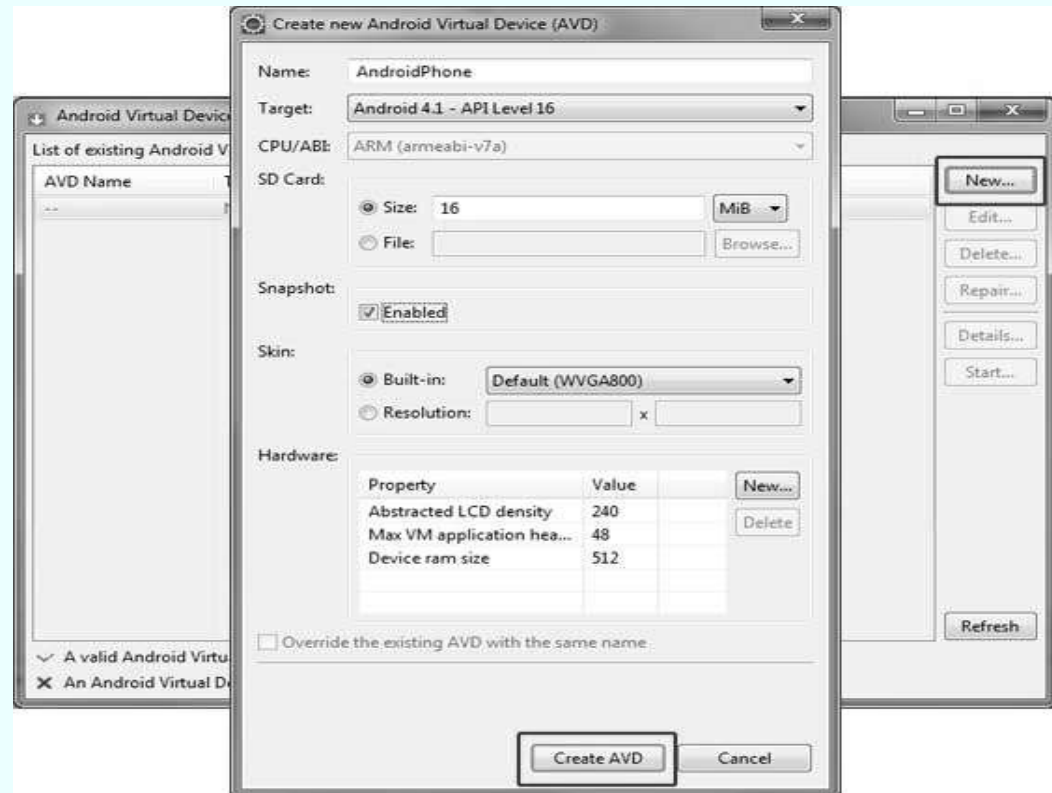
- Set name = ADT plugin
- Location = <https://dlssl.google.com/android/eclipse/>
- Select All and click install



Setup (7)

Android virtual devices:

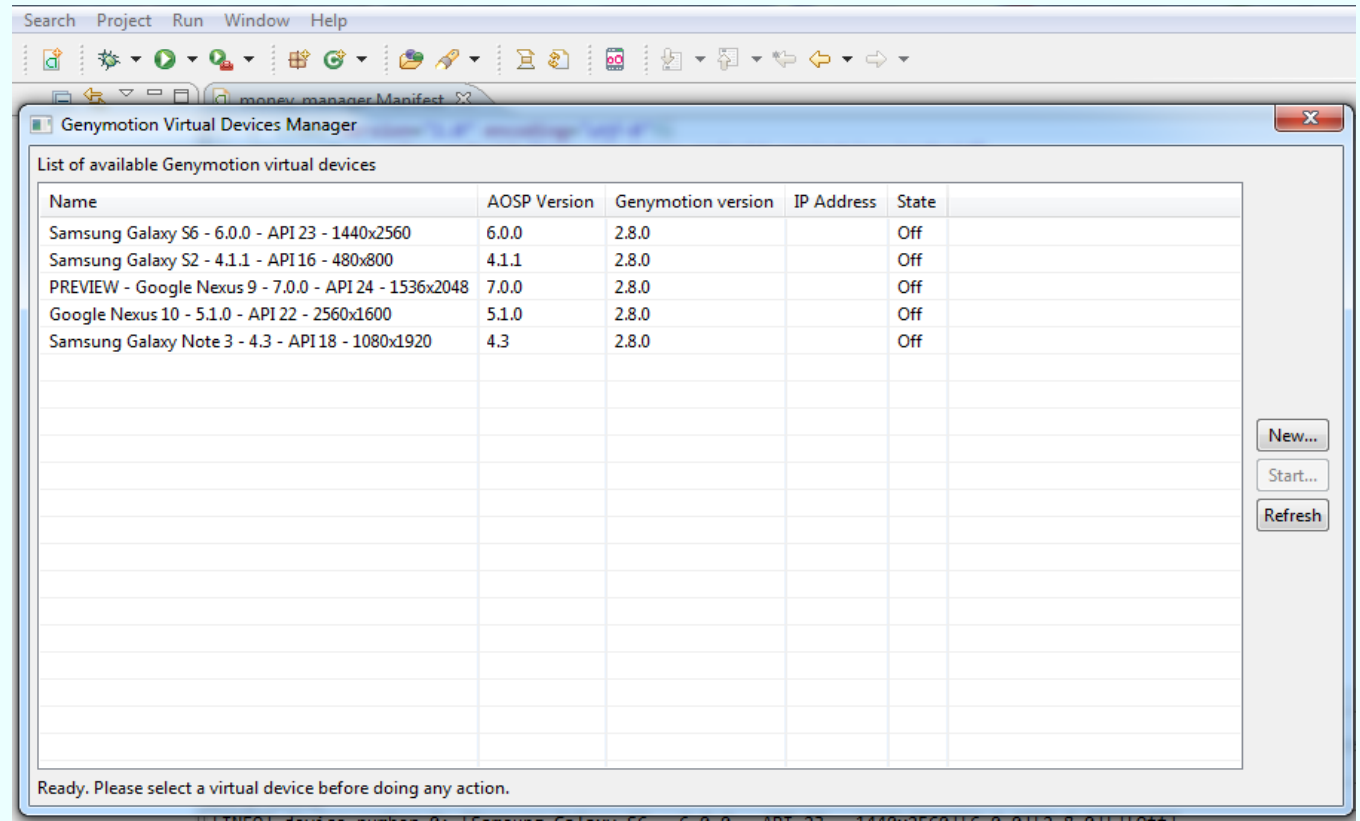
- In Eclipse: Window → AVD Manager → to run AVD
- Click new to add new device

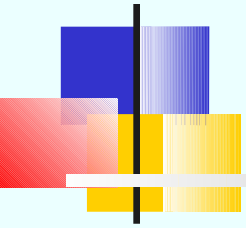


Setup (8)

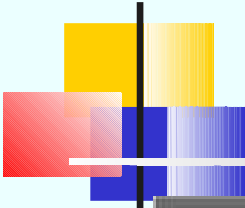
Gennymotion:

- In Eclipse: Click on the plugged icon of Gennymotion
- Click add new to create new virtual device

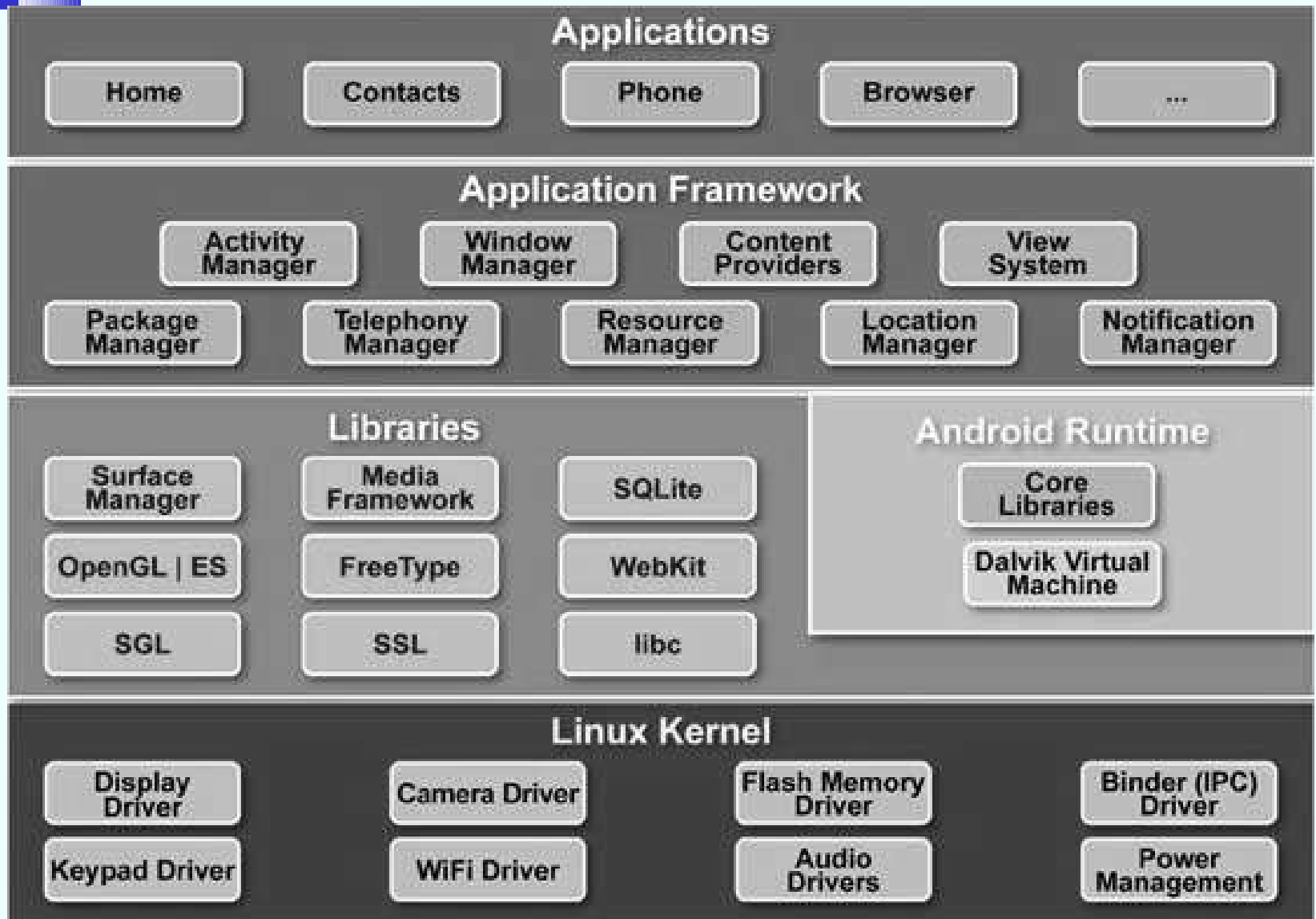


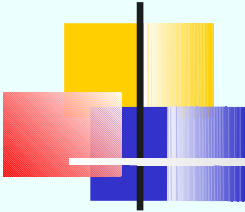


Application Components



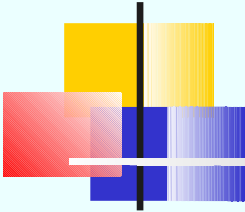
Android Architect





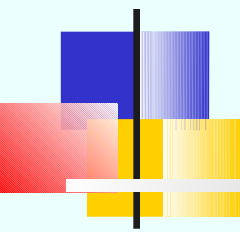
Main components

Component	Description
Activities	They dictate the UI and handle the user interaction to the smartphone screen
Services	They handle background processing associated with an application.
Broadcast receivers	They handle communication between Android OS and applications.
Content providers	They handle data and database management issues.

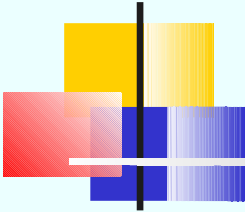


Additional components

Component	Description
Fragments	Represent a behavior or a portion of user interface in an Activity.
Views	UI elements that are drawn onscreen including buttons, lists forms etc.
Layouts	View hierarchies that control screen format and appearance of the views.
Intents	Messages wiring components together.
Resources	External elements, such as strings, constants and drawable pictures.
Manifest	Configuration file for the application.



HelloWorld example



Create app

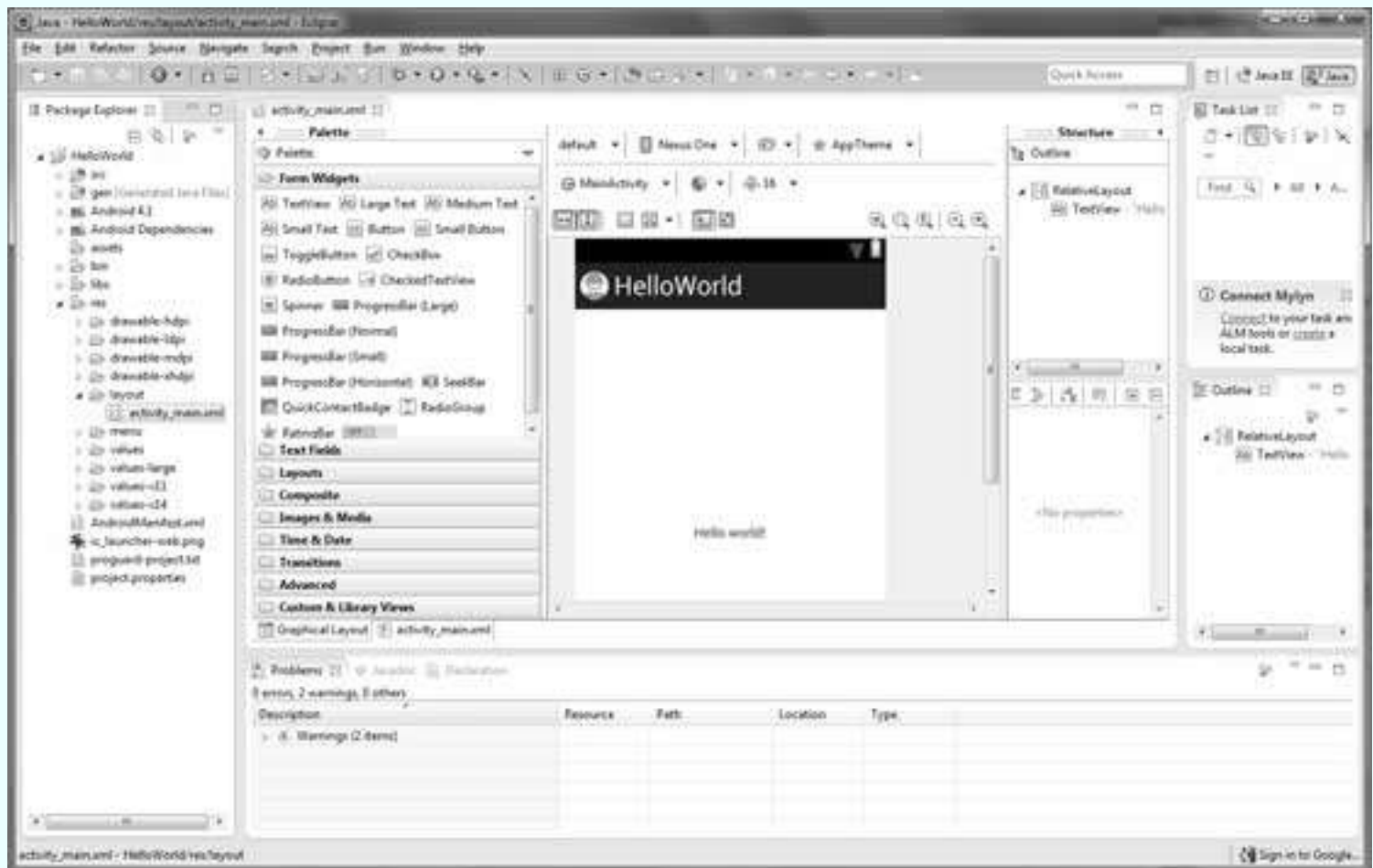
In Eclipse:

- File → new → project → New Android Application
- Set name = helloworld



Editor

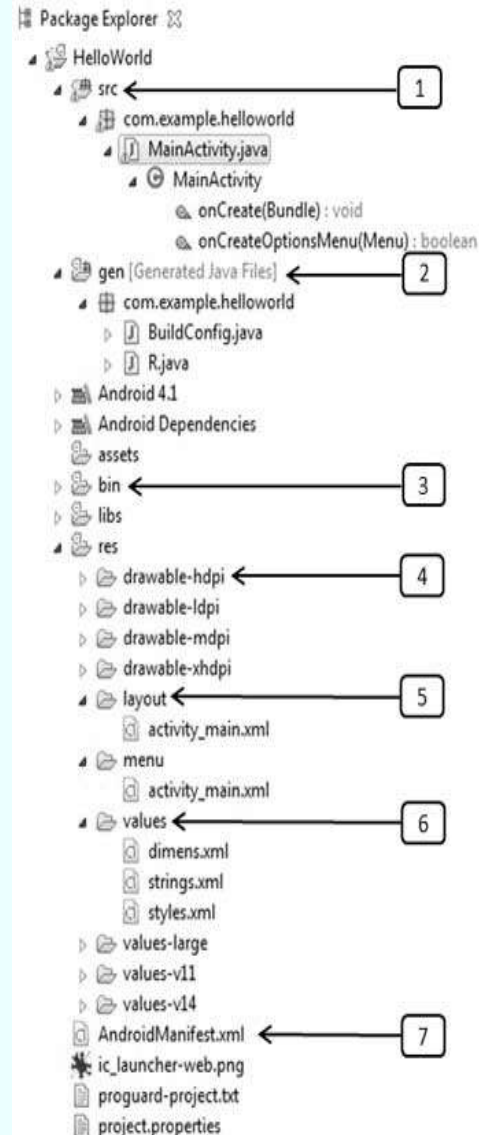
In Eclipse:



Anatomy (1)

In Eclipse:

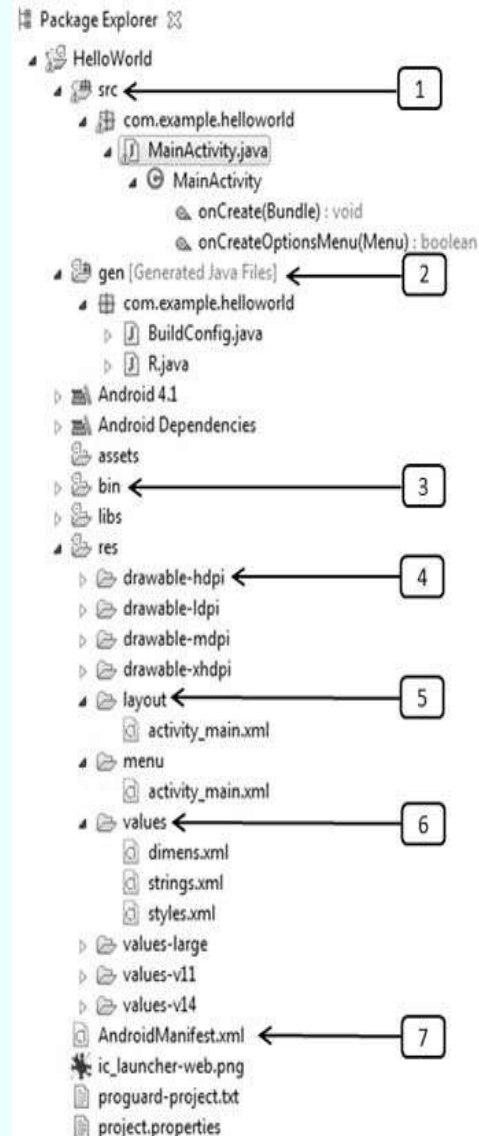
- **src:** contains the **.java** source files
- **gen:** contains the **.R** file
- **bin:** contains the Android package files **.apk** built by the ADT
- **res/drawable-hdpi:** for drawable objects

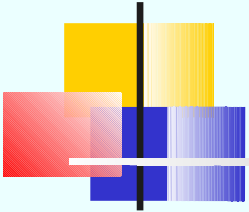


Anatomy (2)

In Eclipse:

- **res/layout:** for files that define your app's user interface
- **res/values:** for other various XML files that contain a collection of resources
- **AndroidManifest.xml:** describes the fundamental characteristics of the app and defines each of its components



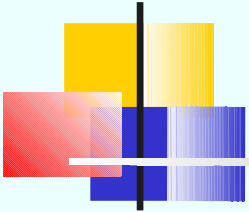


MainActivity.java

```
package com.example.helloworld;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.MenuItem;
import android.support.v4.app.NavUtils;

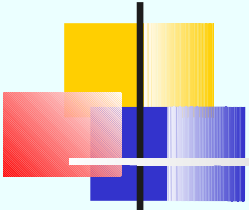
public class MainActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.activity_main, menu);
        return true;
    }
}
```



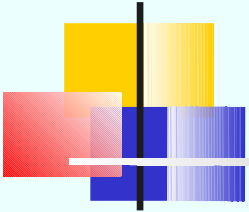
AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.example.helloworld"
android:versionCode="1" android:versionName="1.0" >
<uses-sdk
    android:minSdkVersion="8"
    android:targetSdkVersion="15" />
<application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <activity
        android:name=".MainActivity"
        android:label="@string/title_activity_main" >
        <intent-filter>
            <action
                android:name="android.intent.action.MAIN" />
            <category
                android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```



res/strings.xml

```
<resources>
    <string name="app_name">HelloWorld</string>
    <string name="hello_world">Hello world!</string>
    <string name="menu_settings">Settings</string>
    <string name="title_activity_main">MainActivity</string>
</resources>
```



res/layout/activity_main.xml

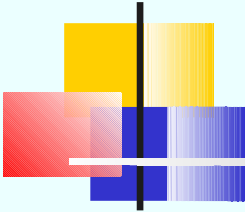
```
<RelativeLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent" >
```

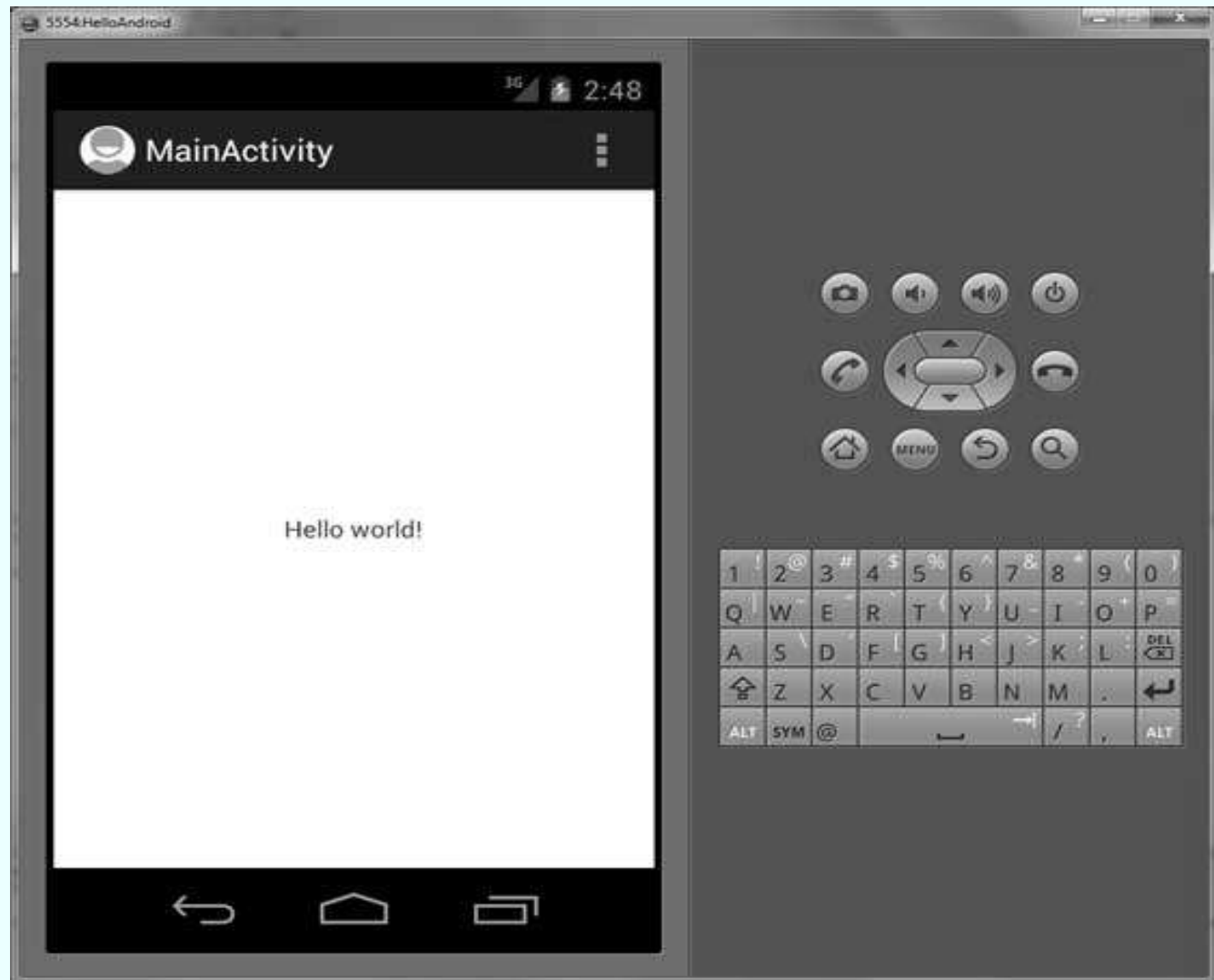
```
    <TextView
```

```
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:layout_centerHorizontal="true"  
        android:layout_centerVertical="true"  
        android:padding="@dimen/padding_medium"  
        android:text="@string/hello_world"  
        tools:context=".MainActivity" />
```

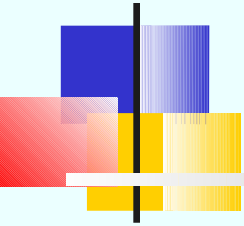
```
</RelativeLayout>
```

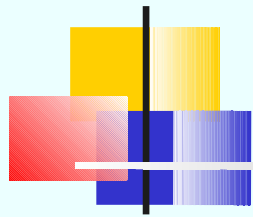


Run the app



Organizing & accessing the resources

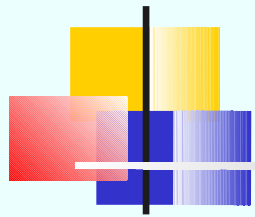




Directory res/

All resources are stored inside the res/:

- **anim/**: xml files that define property animations
- **color/**: xml files that define a state list of colors
- **drawable/**: all image files
- **layout/**: xml files that define a user interface layout
- **menu/**: xml files that define application menus
- **raw/**: Arbitrary files to save in their raw form
- **values/**: xml files that contain simple values, such as strings, integers, and colors.
- **xml/**: Arbitrary XML files that can be read at runtime by calling `Resources.getXML()`



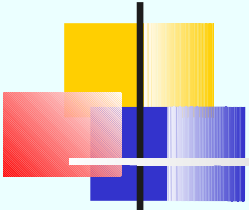
Accessing res/ in code (1)

res/values/strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello">Hello, World!</string>
</resources>
```

MainActivity.java:

```
TextView msgTextView = (TextView) findViewById(R.id.msg);
msgTextView.setText(R.string.hello);
```



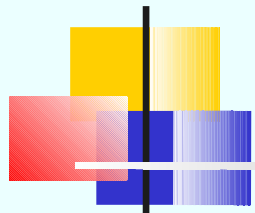
Accessing res/ in code (2)

res/layout/activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />

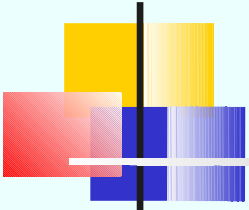
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
</LinearLayout>
```



Accessing res/ in code (3)

MainActivity.java:

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.main_activity);  
  
    TextView txt = (TextView)findViewById(R.id.text);  
    Button btn = (Button)findViewById(R.id.button);  
}
```



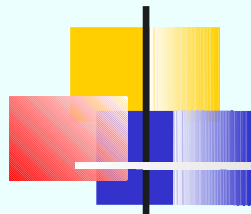
Accessing res/ in XML

res/values/strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="opaque_red">#f00</color>
    <string name="hello">Hello!</string>
</resources>
```

res/layout/activity_main.xml:

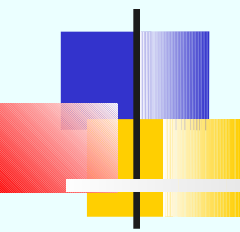
```
<?xml version="1.0" encoding="utf-8"?>
<EditText
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:textColor="@color/opaque_red"
    android:text="@string/hello" />
```



Exercises

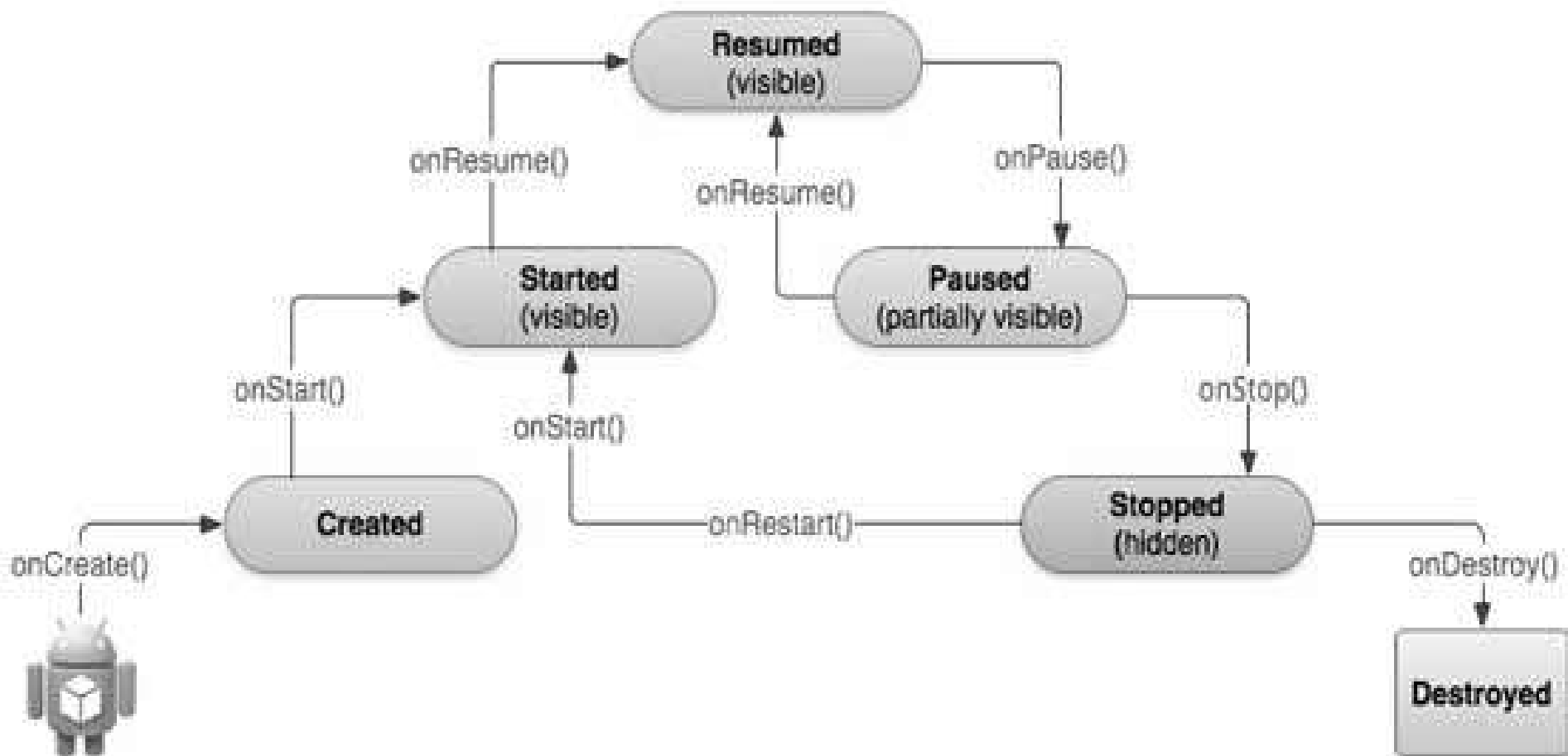
Within the helloworld example:

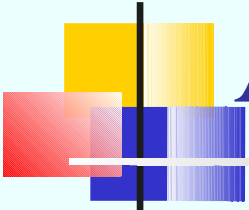
- Change the value of `hello_world` in `strings.xml` to see what happens
- Change the position alignment of the text on the screen by modifying the `activity_main.xml`
- How to change the color of the text?



Activities

Activity life cycle (1)

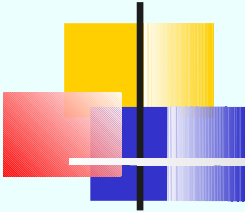




Activity life cycle (2)

States of an activity:

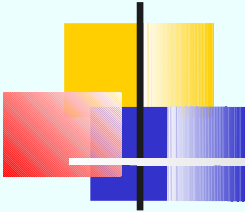
- **onCreate()**: the first callback and called when the activity is first created.
- **onStart()**: called when the activity becomes visible to the user.
- **onResume()**: called when the user starts interacting with the application.
- **onPause()**: called when the current activity is being paused and the previous activity is being resumed.
- **onStop()**: called when the activity is no longer visible.
- **onDestroy()**: called before the activity is destroyed by the system
- **onRestart()**: called when the activity restarts after stopping it.



Example (1)

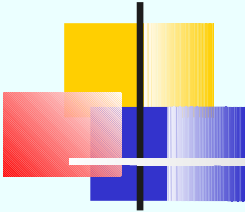
With helloworld, modify the MainActivity.java:

```
package com.example.helloworld;
import android.os.Bundle;
import android.app.Activity;
import android.util.Log;
public class MainActivity extends Activity {
    String msg = "Android : ";
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(msg, "The onCreate() event");
    }
    /** Called when the activity is about to become visible. */
    @Override
    protected void onStart() {
        super.onStart();
        Log.d(msg, "The onStart() event");
    }
}
```



Example (2)

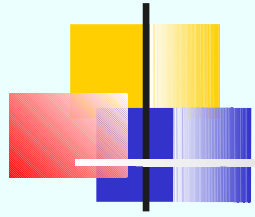
```
/** Called when the activity has become visible. */
@Override
protected void onResume() {
    super.onResume();
    Log.d(msg, "The onResume() event");
}
/** Called when another activity is taking focus. */
@Override
protected void onPause() {
    super.onPause();
    Log.d(msg, "The onPause() event");
}
/** Called when the activity is no longer visible. */
@Override
protected void onStop() {
    super.onStop();
    Log.d(msg, "The onStop() event");
}
/** Called just before the activity is destroyed. */
@Override
public void onDestroy() {
    super.onDestroy();
    Log.d(msg, "The onDestroy() event");
}
```



Example (3)

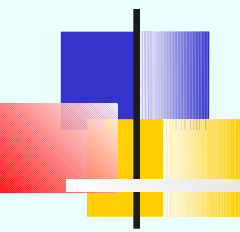
AndroidManifest.xml:

```
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.helloworld"
  android:versionCode="1"
  android:versionName="1.0" >
  <uses-sdk
    android:minSdkVersion="8"
    android:targetSdkVersion="15" />
  <application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <activity
      android:name=".MainActivity"
      android:label="@string/title_activity_main" >
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category
          android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
      </activity>
    </application>
</manifest>
```



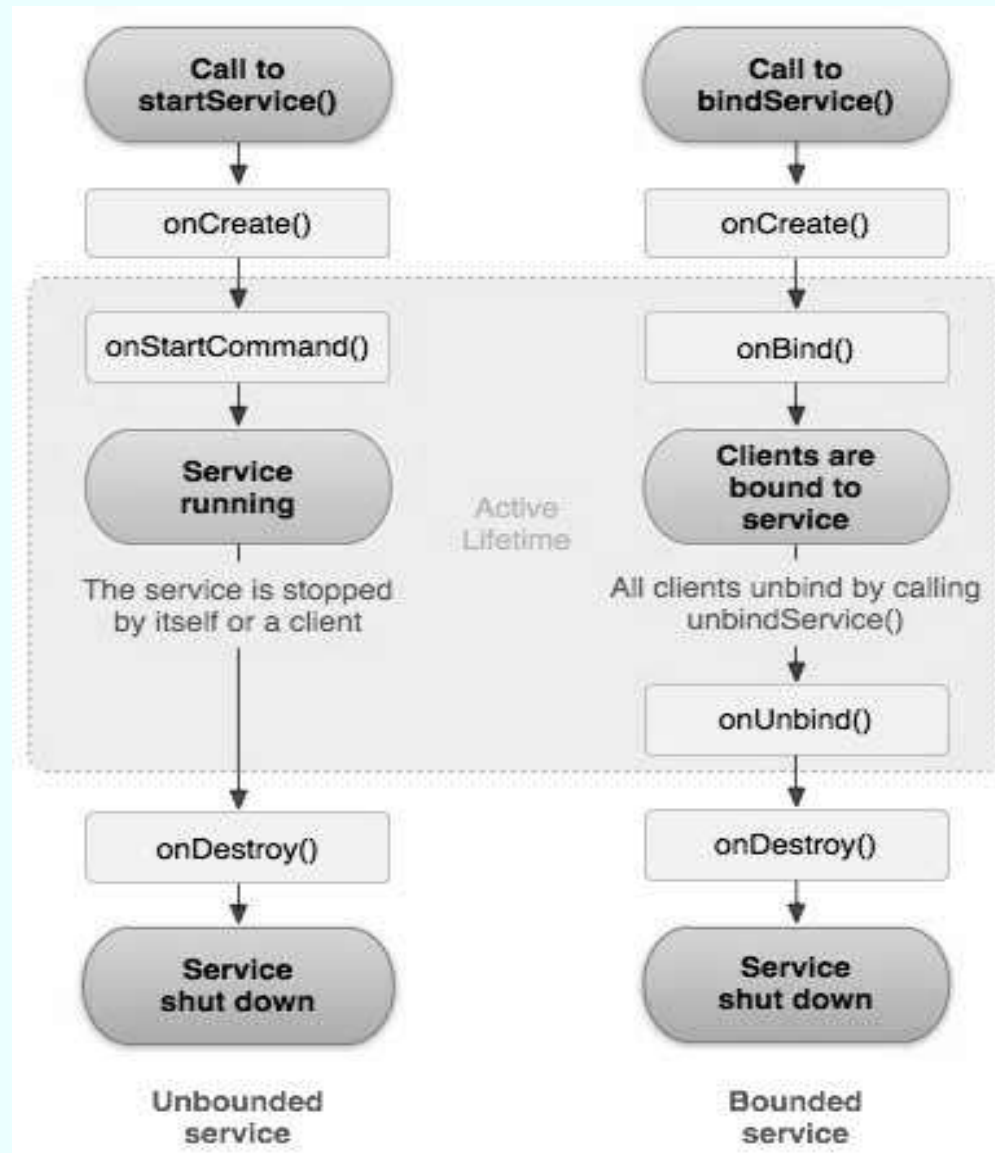
Example (4)

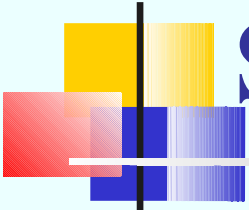
Run the exampe and observe the result in log tab.



Services

Service life cycle (1)





Service life cycle (2)

States of a service:

- **onStartCommand()**: called when service be started, by calling *startService()*
- **onBind()**: called when another component wants to bind with the service by calling *bindService()*.
- **onUnbind()**: called when all clients have disconnected from a particular interface published by the service.
- **onRebind()**: called when new clients have connected to the service, after it had previously been notified that all had disconnected in its *onUnbind(Intent)*.
- **onCreate()**: called when the service is first created using *onStartCommand()* or *onBind()*.
- **onDestroy()**: called when the service is no longer used and is being destroyed.



Example (1)

With helloworld, modify the MainActivity.java:

```
public class MainActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.activity_main, menu);
        return true;
    }
    // Method to start the service
    public void startService(View view) {
        startService(new Intent(getBaseContext(), MyService.class));
    }
    // Method to stop the service
    public void stopService(View view) {
        stopService(new Intent(getBaseContext(), MyService.class));
    }
}
```



Example (2)

With helloworld, add the MyService.java:

```
package com.example.helloworld;
import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;
public class MyService extends Service {
    @Override
    public IBinder onBind(Intent arg0) {
        return null;
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId){
        // Let it continue running until it is stopped.
        Toast.makeText(this, "Service Started",
            Toast.LENGTH_LONG).show();
        return START_STICKY;
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        Toast.makeText(this, "Service Destroyed",
            Toast.LENGTH_LONG).show();
    }
}
```



Example (3)

AndroidManifest.xml:

```
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.helloworld"
  android:versionCode="1"
  android:versionName="1.0" >
  <uses-sdk
    android:minSdkVersion="8"
    android:targetSdkVersion="15" />
  <application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <activity
      android:name=".MainActivity"
      android:label="@string/title_activity_main" >
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category
          android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <service android:name=".MyService" />
  </application>
</manifest>
```



Example (4)

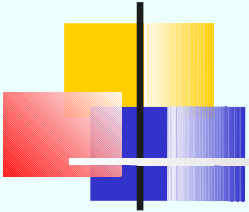
res/layout/activity_main.xml:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <Button android:id="@+id/btnStartService"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/start_service"
        android:onClick="startService"/>

    <Button android:id="@+id/btnStopService"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/stop_service"
        android:onClick="stopService" />

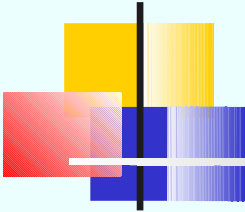
</LinearLayout>
```



Example (5)

res/values/strings.xml:

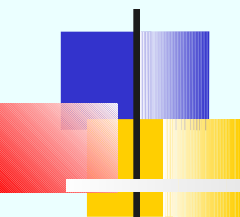
```
<resources>
<string name="app_name">HelloWorld</string>
<string name="hello_world">Hello world!</string>
<string name="menu_settings">Settings</string>
<string name="title_activity_main">MainActivity</string>
<string name="start_service">Start Service</string>
<string name="stop_service">Stop Service</string>
</resources>
```



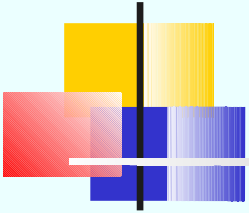
Example (6)

Run the example and observe the result.





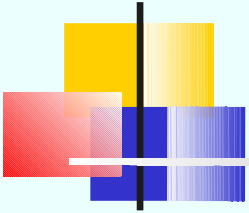
Broadcast receivers



Create

Create a subclass of BroadcastReceiver:

```
public class MyReceiver extends BroadcastReceiver {  
  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        Toast.makeText(context, "Intent Detected.",  
            Toast.LENGTH_LONG).show();  
    }  
}
```

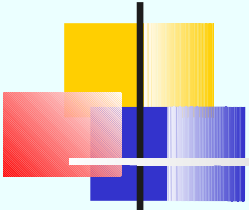


Registering

Add content in AndroidManifest.xml:

```
<application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >

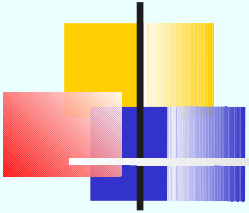
    <receiver android:name="MyReceiver">
        <intent-filter>
            <action android:name="android.intent.action.BOOT_COMPLETED">
            </action>
        </intent-filter>
    </receiver>
</application>
```



Broadcast events

Constant in android.intent.action:

- BATTERY_CHANGED
- BATTERY_LOW
- BATTERY_OKAY
- BOOT_COMPLETED
- BUG_REPORT
- CALL
- CALL_BUTTON
- DATE_CHANGED
- REBOOT

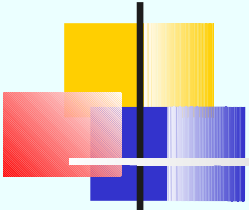


Broadcasting custom event (1)

Add content in AndroidManifest.xml:

```
<application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >

    <receiver android:name="MyReceiver">
        <intent-filter>
            <action android:name="com.tutorialspoint.CUSTOM_INTENT">
            </action>
        </intent-filter>
    </receiver>
</application>
```



Broadcasting custom event (2)

In the code:

```
public void broadcastIntent(View view) {  
    Intent intent = new Intent();  
    intent.setAction("com.tutorialspoint.CUSTOM_INTENT");  
    sendBroadcast(intent);  
}
```



Example (1)

MainActivity.java:

```
package com.example.helloworld;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.content.Intent;
import android.view.View;
public class MainActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.activity_main, menu);
        return true;
    }
    // broadcast a custom intent.
    public void broadcastIntent(View view){
        Intent intent = new Intent();
        intent.setAction("com.tutorialspoint.CUSTOM_INTENT");
        sendBroadcast(intent);
    }
}
```



Example (2)

MyReceiver.java:

```
package com.example.helloworld;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class MyReceiver extends BroadcastReceiver {

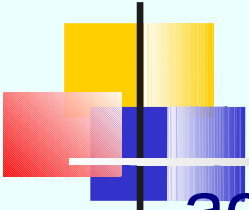
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Intent Detected.",
            Toast.LENGTH_LONG).show();
    }
}
```



Example (3)

AndroidManifest.xml:

```
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.helloworld"
  android:versionCode="1"
  android:versionName="1.0" >
  <uses-sdk
    android:minSdkVersion="8"
    android:targetSdkVersion="15" />
  <application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <activity
      android:name=".MainActivity"
      android:label="@string/title_activity_main" >
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category
          android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
      </activity>
      <receiver android:name="MyReceiver">
        <intent-filter>
          <action android:name="com.tutorialspoint.CUSTOM_INTENT">
          </action>
        </intent-filter>
      </receiver>
    </application>
  </manifest>
```



Example (4)

activity_main.xml:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <Button android:id="@+id/btnStartService"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/broadcast_intent"
        android:onClick="broadcastIntent"/>

</LinearLayout>
```



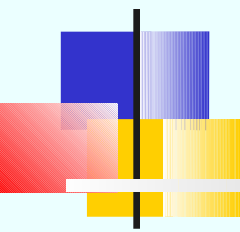
Example (5)

strings.xml:

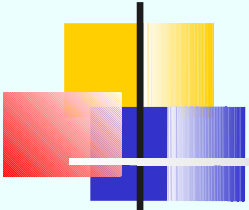
```
<resources>
    <string name="app_name">HelloWorld</string>
    <string name="hello_world">Hello world!</string>
    <string name="menu_settings">Settings</string>
    <string name="title_activity_main">MainActivity</string>
    <string name="broadcast_intent">Broadcast Intent</string>
</resources>
```

Example (6)





Content provider



Content URI

To query a content provider:

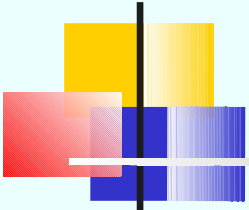
`<prefix>://<authority>/<data_type>/<id>`

prefix: is always set to content://

authority: specifies the name of the content provider

data_type: the type of data that this particular provider provides

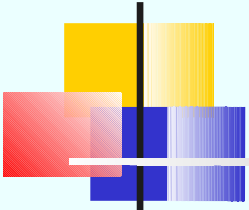
id: specifies the specific record requested



Create content provider

Step by step:

- create a Content Provider class that extends the *ContentProviderbase* class.
- define content provider URI address
- create a database to keep the content.
- implement Content Provider queries to perform different database specific operations.
- register the Content Provider in your activity file using <provider> tag



Example (1)

Step by step:

- create an Android application and name it as *MyContentProvider* under a package *com.example.mycontentprovider*, with blank Activity.
- In *MainActivity.java*, add two new methods *onClickAddName()* and *onClickRetrieveStudents()*.
- Create a new java file called *StudentsProvider.java*
- Register content provider in *AndroidManifest.xml*
- Modify the default content of *res/layout/activity_main.xml*
- Define required constants in *res/values/strings.xml*



Example (2)

MainActivity.java (1):

```
package com.example.mycontentprovider;
import android.net.Uri;
import android.os.Bundle;
import android.app.Activity;
import android.content.ContentValues;
import android.content.CursorLoader;
import android.database.Cursor;
import android.view.Menu;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```

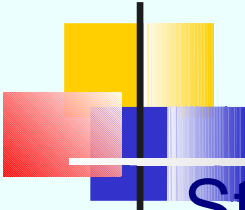


Example (3)

MainActivity.java (2):

```
public void onClickAddName(View view) {
    // Add a new student record
    ContentValues values = new ContentValues();
    values.put(StudentsProvider.NAME,
        ((EditText)findViewById(R.id.txtName)).getText().toString());
    values.put(StudentsProvider.GRADE,
        ((EditText)findViewById(R.id.txtGrade)).getText().toString());
    Uri uri = getContentResolver().insert(
        StudentsProvider.CONTENT_URI, values);
    Toast.makeText(getBaseContext(), uri.toString(),
        Toast.LENGTH_LONG).show();
}

public void onClickRetrieveStudents(View view) {
    // Retrieve student records
    String URL = "content://com.example.provider.College/students";
    Uri students = Uri.parse(URL);
    Cursor c = managedQuery(students, null, null, null, "name");
    if (c.moveToFirst()) {
        do{
            Toast.makeText(this, c.getString(c.getColumnIndex(
                StudentsProvider.ID)) + ", " +
                c.getString(c.getColumnIndex(StudentsProvider.NAME)) +
                ", " + c.getString(c.getColumnIndex(
                StudentsProvider.GRADE)), Toast.LENGTH_SHORT).show();
        } while (c.moveToNext());
    }
}
```



Example (4)

StudentProvider.java (1):

```
package com.example.mycontentprovider;
import java.util.HashMap;
import android.content.ContentProvider;
import android.content.ContentUris;
import android.content.ContentValues;
import android.content.Context;
import android.content.UriMatcher;
import android.database.Cursor;
import android.database.SQLException;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.database.sqlite.SQLiteQueryBuilder;
import android.net.Uri;
import android.text.TextUtils;

public class StudentsProvider extends ContentProvider {
    static final String PROVIDER_NAME = "com.example.provider.College";
    static final String URL = "content://" + PROVIDER_NAME + "/student";
    static final Uri CONTENT_URI = Uri.parse(URL);
    static final String _ID = "id";
    static final String _NAME = "name";
    static final String _GRADE = "grade";
    private static HashMap<String, String> STUDENTS_PROJECTION_MAP;
    static final int STUDENTS = 1;
    static final int STUDENT_ID = 2;
    static final UriMatcher uriMatcher;
```



Example (5)

StudentProvider.java (2):

```
static{
    uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
    uriMatcher.addURI(PROVIDER_NAME, "students", STUDENTS);
    uriMatcher.addURI(PROVIDER_NAME, "students/#", STUDENT_ID);
}
private SQLiteDatabase db;
static final String DATABASE_NAME = "College";
static final String STUDENTS_TABLE_NAME = "students";
static final int DATABASE_VERSION = 1;
static final String CREATE_DB_TABLE = " CREATE TABLE " +
    STUDENTS_TABLE_NAME + " (_id INTEGER PRIMARY KEY AUTOINCREMENT, "
    " name TEXT NOT NULL, grade TEXT NOT NULL);"

private static class DatabaseHelper extends SQLiteOpenHelper {

    DatabaseHelper(Context context){
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }
    @Override
    public void onCreate(SQLiteDatabase db){
        db.execSQL(CREATE_DB_TABLE);
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion,
        int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + STUDENTS_TABLE_NAME);
        onCreate(db);
    }
}
```



Example (6)

StudentProvider.java (3):

```
@Override
public boolean onCreate() {
    Context context = getContext();
    DatabaseHelper dbHelper = new DatabaseHelper(context);
    db = dbHelper.getWritableDatabase();
    return (db == null)? false:true;
}

@Override
public Uri insert(Uri uri, ContentValues values) {
    long rowID = db.insert( STUDENTS_TABLE_NAME, "", values);
    if (rowID > 0){
        Uri _uri = ContentUris.withAppendedId(CONTENT_URI, rowID);
        getContext().getContentResolver().notifyChange(_uri, null);
        return _uri;
    }
    throw new SQLException("Failed to add a record into " + uri);
}
```



Example (7)

StudentProvider.java (4):

```
@Override
public Cursor query(Uri uri, String[] projection, String selection,
String[] selectionArgs, String sortOrder) {
    SQLiteQueryBuilder qb = new SQLiteQueryBuilder();
    qb.setTables(STUDENTS_TABLE_NAME);
    switch (uriMatcher.match(uri)) {
        case STUDENTS:
            qb.setProjectionMap(STUDENTS_PROJECTION_MAP);
            break;
        case STUDENT_ID:
            qb.appendWhere( "_ID + "=" + uri.getPathSegments().get(1))";
            break;
        default:
            throw new IllegalArgumentException("Unknown URI " + uri);
    }
    if (sortOrder == null || sortOrder == ""){
        sortOrder = NAME;
    }
    Cursor c = qb.query(db, projection, selection, selectionArgs,
        null, null, sortOrder);
    c.setNotificationUri(getContext().getContentResolver(), uri);
    return c;
}
```



Example (8)

StudentProvider.java (5):

```
@Override
public int delete(Uri uri, String selection, String[] selectionArgs) {
    int count = 0;
    switch (uriMatcher.match(uri)) {
        case STUDENTS:
            count = db.delete(STUDENTS_TABLE_NAME, selection,
                             selectionArgs);
            break;
        case STUDENT_ID:
            String id = uri.getPathSegments().get(1);
            count = db.delete(STUDENTS_TABLE_NAME, _ID + " = " + id +
                             (!TextUtils.isEmpty(selection) ? " AND (" +
                             selection + ')' : ""), selectionArgs);
            break;
        default:
            throw new IllegalArgumentException("Unknown URI " + uri);
    }
    getContext().getContentResolver().notifyChange(uri, null);
    return count;
}
```



Example (9)

StudentProvider.java (6):

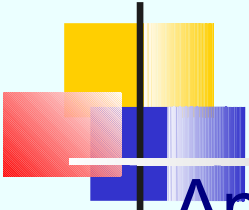
```
@Override
public int update(Uri uri, ContentValues values, String selection,
String[] selectionArgs) {
    int count = 0;
    switch (uriMatcher.match(uri)) {
        case STUDENTS:
            count = db.update(STUDENTS_TABLE_NAME, values,
                selection, selectionArgs);
            break;
        case STUDENT_ID:
            count = db.update(STUDENTS_TABLE_NAME, values, _ID +
                " = " + uri.getPathSegments().get(1) +
                (!TextUtils.isEmpty(selection) ? " AND (" +
                    selection + ')' : ""), selectionArgs);
            break;
        default:
            throw new IllegalArgumentException("Unknown URI " + uri);
    }
    getContext().getContentResolver().notifyChange(uri, null);
    return count;
}
```



Example (10)

StudentProvider.java (7):

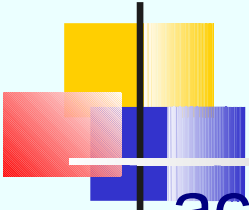
```
@Override
public String getType(Uri uri) {
    switch (uriMatcher.match(uri)){
        case STUDENTS:
            return "vnd.android.cursor.dir/vnd.example.students";
        case STUDENT_ID:
            return "vnd.android.cursor.item/vnd.example.students";
        default:
            throw new IllegalArgumentException(
                "Unsupported URI: " + uri);
    }
}
```



Example (11)

AndroidManifest.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.mycontentprovider"
    android:versionCode="1"
    android:versionName="1.0" >
    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="17" />
    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name="com.example.mycontentprovider.MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <provider android:name="StudentsProvider"
            android:authorities="com.example.provider.College">
        </provider>
    </application>
</manifest>
```



Example (12)

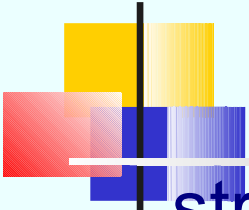
activity_main.xml:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Name" />
    <EditText android:id="@+id/txtName"
        android:layout_height="wrap_content"
        android:layout_width="fill_parent" />
    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Grade" />
    <EditText android:id="@+id/txtGrade"
        android:layout_height="wrap_content"
        android:layout_width="fill_parent" />

    <Button android:text="Add Name"
        android:id="@+id/btnAdd"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:onClick="onClickAddName" />
    <Button android:text="Retrieve Students"
        android:id="@+id/btnRetrieve"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:onClick="onClickRetrieveStudents" />

</LinearLayout>
```

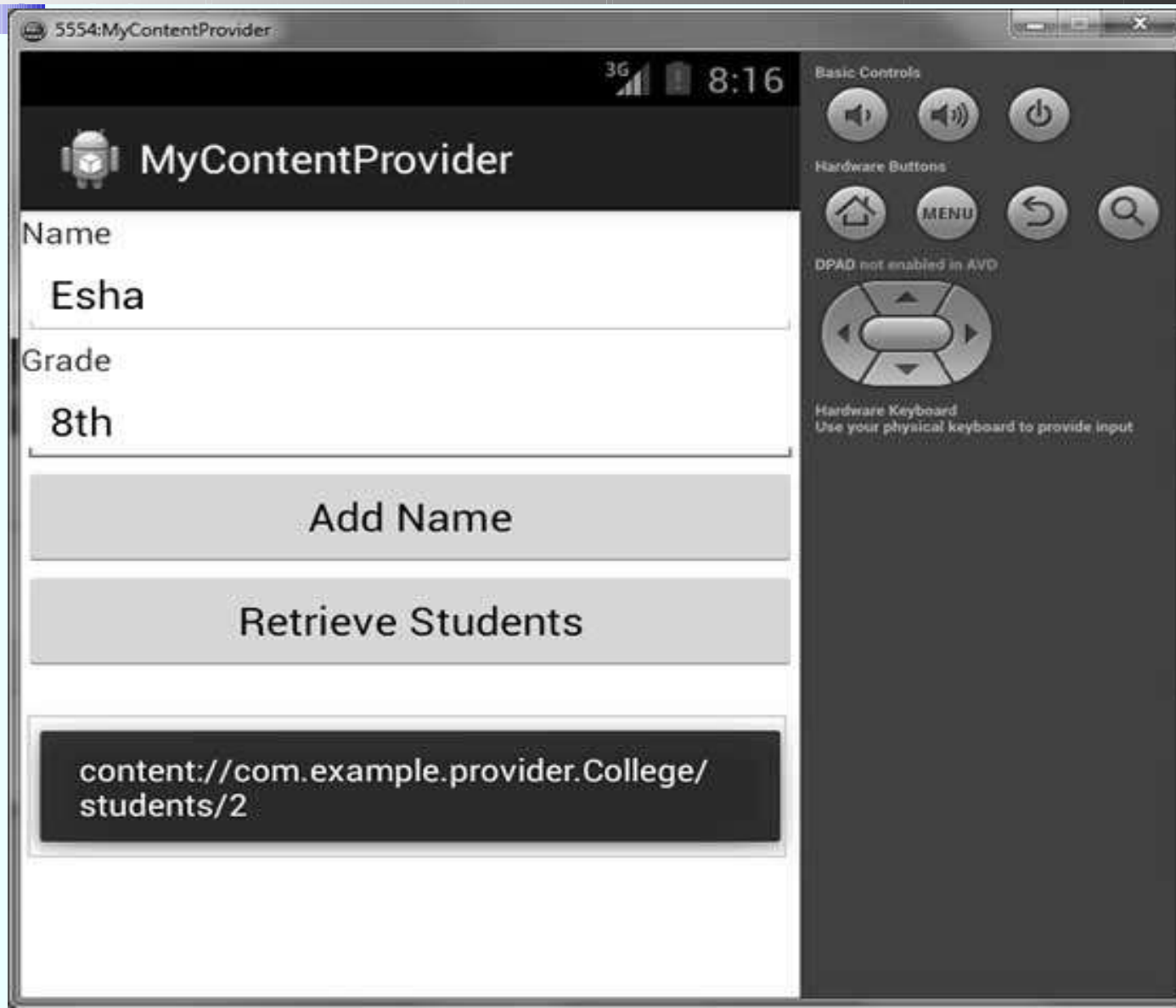


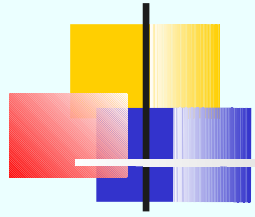
Example (13)

strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">MyContentProvider</string>
    <string name="action_settings">Settings</string>
    <string name="hello_world">Hello world!</string>
</resources>;
```

Example (14)

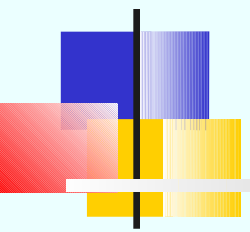




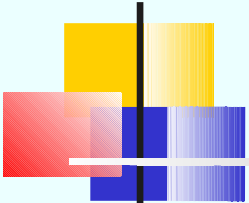
Exercises

With the student management example:

- Change the app become a local database
- Add some functions: edit, delete a student



SQLite



SQLite (1)

Create:

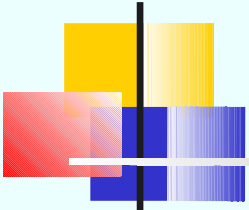
```
SQLiteDatabase mydatabase = openOrCreateDatabase("your database  
name", MODE_PRIVATE, null);
```

Insertion:

```
mydatabase.execSQL("CREATE TABLE IF NOT EXISTS  
    Tutorialspoint(Username VARCHAR,Password VARCHAR);");  
mydatabase.execSQL("INSERT INTO Tutorialspoint  
    VALUES('admin','admin');");
```

fetching:

```
Cursor resultSet = mydatabase.rawQuery(  
    "Select * from Tutorialspoint",null);  
resultSet.moveToFirst();  
String username = resultSet.getString(1);  
String password = resultSet.getString(2);
```



SQLite (2)

Helper class:

```
public class DBHelper extends SQLiteOpenHelper {  
    public DBHelper() {  
        super(context, DATABASE_NAME, null, 1);  
    }  
    public void onCreate(SQLiteDatabase db) {}  
    public void onUpgrade(SQLiteDatabase database,  
        int oldVersion, int newVersion) {}  
}
```



Example (1)

MainActivity.java (1):

```
public class MainActivity extends Activity {
    public final static String EXTRA_MESSAGE =
        "com.example.AddressBook.MESSAGE";
    private ListView obj;
    DBHelper mydb;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mydb = new DBHelper(this);
        ArrayList array_list = mydb.getAllCotacts();
        ArrayAdapter arrayAdapter = new ArrayAdapter(this,
            android.R.layout.simple_list_item_1, array_list);
        //adding it to the list view.
        obj = (ListView)findViewById(R.id.listView1);
        obj.setAdapter(arrayAdapter);
        obj.setOnItemClickListener(new OnItemClickListener() {
            @Override
            public void onItemClick(AdapterView<?> arg0, View arg1,
                int arg2, long arg3) {
                // TODO Auto-generated method stub
                int id To Search = arg2 + 1;
                Bundle dataBundle = new Bundle();
                dataBundle.putInt("id", id To Search);
                Intent intent = new Intent(getApplicationContext(),
                    com.example.addressbook.DisplayContact.class);
                intent.putExtras(dataBundle);
                startActivity(intent);
            }
        });
    }
}
```



Example (2)

MainActivity.java (2):

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    super.onOptionsItemSelected(item);
    switch(item.getItemId()) {
        case R.id.item1:
            Bundle dataBundle = new Bundle();
            dataBundle.putInt("id", 0);
            Intent intent = new Intent(getApplicationContext(),
                com.example.addressbook.DisplayContact.class);
            intent.putExtras(dataBundle);
            startActivity(intent);
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}

public boolean onKeyDown(int keycode, KeyEvent event) {
    if (keycode == KeyEvent.KEYCODE_BACK) {
        moveTaskToBack(true);
    }
    return super.onKeyDown(keycode, event);
}
```



Example (3)

DisplayContact.java (1):

```
public class DisplayContact extends Activity {
    int from Where_I_Am_Coming = 0;
    private DBHelper mydb ;
    TextView name ;
    TextView phone;
    TextView email;
    TextView street;
    TextView place;
    int id_To_Update = 0;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_display_contact);
        name = (TextView) findViewById(R.id.editTextName);
        phone = (TextView) findViewById(R.id.editTextPhone);
        email = (TextView) findViewById(R.id.editTextStreet);
        street = (TextView) findViewById(R.id.editTextEmail);
        place = (TextView) findViewById(R.id.editTextCity);
        mydb = new DBHelper(this);
    }
}
```



Example (4)

DisplayContact.java (2):

```
Bundle extras = getIntent().getExtras();
if(extras !=null){
    int Value = extras.getInt("id");
    if(Value>0){
        //means this is the view part not the add contact part.
        Cursor rs = mydb.getData(Value);
        id To Update = Value;
        rs.moveToFirst();
        String nam =rs.getString(rs.getColumnIndex(
            DBHelper.CONTACTS_COLUMN_NAME));
        String phon = rs.getString(rs.getColumnIndex(
            DBHelper.CONTACTS_COLUMN_PHONE));
        String emai = rs.getString(rs.getColumnIndex(
            DBHelper.CONTACTS_COLUMN_EMAIL));
        String stree = rs.getString(rs.getColumnIndex(
            DBHelper.CONTACTS_COLUMN_STREET));
        String plac = rs.getString(rs.getColumnIndex(
            DBHelper.CONTACTS_COLUMN_CITY));
        if (!rs.isClosed()){
            rs.close();
        }
        Button b = (Button)findViewById(R.id.button1);
        b.setVisibility(View.INVISIBLE);
        name.setText((CharSequence)nam);
        name.setFocusable(false); name.setClickable(false);
        phone.setText((CharSequence)phon);
        phone.setFocusable(false); phone.setClickable(false);
        email.setText((CharSequence)emai);
        email.setFocusable(false); email.setClickable(false);
        street.setText((CharSequence)stree);
        street.setFocusable(false); street.setClickable(false);
        place.setText((CharSequence)plac);
        place.setFocusable(false); place.setClickable(false);
    }
}
```

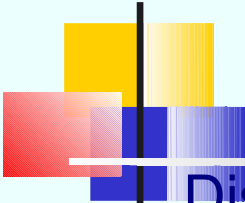


Example (5)

DisplayContact.java (3):

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    Bundle extras = getIntent().getExtras();
    if(extras !=null){
        int Value = extras.getInt("id");
        if(Value>0){
            getMenuInflater().inflate(R.menu.display_contact, menu);
        }else{
            getMenuInflater().inflate(R.menu.main, menu);
        }
    }
    return true;
}

public boolean onOptionsItemSelected(MenuItem item){
    super.onOptionsItemSelected(item);
    switch(item.getItemId()){
        case R.id.Edit Contact:
            Button b =(Button)findViewById(R.id.button1);
            b.setVisibility(View.VISIBLE);
            name.setEnabled(true);      name.setFocusableInTouchMode(true);
            name.setClickable(true);   phone.setEnabled(true);
            phone.setFocusableInTouchMode(true);
            phone.setClickable(true);  email.setEnabled(true);
            email.setFocusableInTouchMode(true);
            email.setClickable(true);  street.setEnabled(true);
            street.setFocusableInTouchMode(true);
            street.setClickable(true); place.setEnabled(true);
            place.setFocusableInTouchMode(true);
            place.setClickable(true);
            return true;
    }
}
```



Example (6)

DisplayContact.java (4):

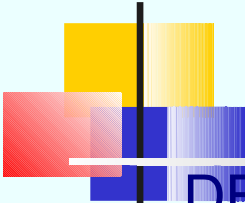
```
case R.id.Delete Contact:
    AlertDialog.Builder builder = new AlertDialog.Builder(this)
    builder.setMessage(R.string.deleteContact)
        .setPositiveButton(R.string.yes, new
            DialogInterface.OnClickListener() {
                public void onClick(DialogInterface dialog, int id) {
                    mydb.deleteContact(id To Update);
                    Toast.makeText(getApplicationContext(), "Deleted
                        Successfully", Toast.LENGTH_SHORT).show();
                    Intent intent = new Intent(getApplicationContext()
                        com.example.addressbook.MainActivity.class);
                    startActivity(intent);
                }
            })
        .setNegativeButton(R.string.no, new
            DialogInterface.OnClickListener() {
                public void onClick(DialogInterface dialog, int id) {
                    // User cancelled the dialog
                }
            })
    AlertDialog d = builder.create();
    d.setTitle("Are you sure");
    d.show();
    return true;
default:
    return super.onOptionsItemSelected(item);
}
```



Example (7)

DisplayContact.java (5):

```
public void run(View view){
    Bundle extras = getIntent().getExtras();
    if(extras !=null){
        int Value = extras.getInt("id");
        if(Value>0){
            if(mydb.updateContact(id To Update,name.getText().toString(),
                phone.getText().toString(), email.getText().toString(),
                street.getText().toString(), place.getText().toString()){
                Toast.makeText(getApplicationContext(), "Updated",
                    Toast.LENGTH_SHORT).show();
                Intent intent = new Intent(getApplicationContext(),
                    com.example.addressbook.MainActivity.class);
                startActivity(intent);
            }else{
                Toast.makeText(getApplicationContext(), "not Updated",
                    Toast.LENGTH_SHORT).show();
            }
        }else{
            if(mydb.insertContact(name.getText().toString(),
                phone.getText().toString(), email.getText().toString(),
                street.getText().toString(), place.getText().toString()){
                Toast.makeText(getApplicationContext(), "done",
                    Toast.LENGTH_SHORT).show();
            }else{
                Toast.makeText(getApplicationContext(), "not done",
                    Toast.LENGTH_SHORT).show();
            }
            Intent intent = new Intent(getApplicationContext(),
                com.example.addressbook.MainActivity.class);
            startActivity(intent);
        }
    }
}
```



Example (8)

DBHelper.java (1):

```
public class DBHelper extends SQLiteOpenHelper {
    public static final String DATABASE_NAME = "MyDBName.db";
    public static final String CONTACTS_TABLE_NAME = "contacts";
    public static final String CONTACTS_COLUMN_ID = "id";
    public static final String CONTACTS_COLUMN_NAME = "name";
    public static final String CONTACTS_COLUMN_EMAIL = "email";
    public static final String CONTACTS_COLUMN_STREET = "street";
    public static final String CONTACTS_COLUMN_CITY = "place";
    public static final String CONTACTS_COLUMN_PHONE = "phone";
    private HashMap hp;

    public DBHelper(Context context){
        super(context, DATABASE_NAME, null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        // TODO Auto-generated method stub
        db.execSQL("create table contacts " +
            "(id integer primary key, name text,phone text,email text,
            Street text,place text)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
        newVersion) {
        // TODO Auto-generated method stub
        db.execSQL("DROP TABLE IF EXISTS contacts");
        onCreate(db);
    }
}
```



Example (9)

DBHelper.java (2):

```
public boolean insertContact (String name, String phone, String
email, String street,String place){
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("name", name);
    contentValues.put("phone", phone);
    contentValues.put("email", email);
    contentValues.put("street", street);
    contentValues.put("place", place);
    db.insert("contacts", null, contentValues);
    return true;
}

public Cursor getData(int id){
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor res = db.rawQuery( "select * from contacts where
        id="+id+"", null );
    return res;
}

public int numberOfRows(){
    SQLiteDatabase db = this.getReadableDatabase();
    int numRows = (int) DatabaseUtils.queryNumEntries(db,
        CONTACTS_TABLE_NAME);
    return numRows;
}
```



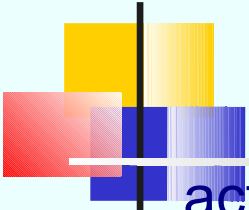
Example (10)

DBHelper.java (3):

```
public boolean updateContact (Integer id, String name, String phone,
String email, String street, String place){
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("name", name);
    contentValues.put("phone", phone);
    contentValues.put("email", email);
    contentValues.put("street", street);
    contentValues.put("place", place);
    db.update("contacts", contentValues, "id = ? ",
        new String[] {Integer.toString(id) } );
    return true;
}

public Integer deleteContact (Integer id){
    SQLiteDatabase db = this.getWritableDatabase();
    return db.delete("contacts", "id = ? ",
        new String[] { Integer.toString(id) } );
}

public ArrayList getAllCotacts(){
    ArrayList array_list = new ArrayList();
    //hp = new HashMap();
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor res = db.rawQuery("select * from contacts", null );
    res.moveToFirst();
    while(res.isAfterLast() == false){
        array_list.add(res.getString(
            res.getColumnIndex(CONTACTS_COLUMN_NAME)));
        res.moveToNext();
    }
    return array_list;
}
```



Example (11)

activity_main.xml:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingBottom="@dimen/activity_vertical_margin"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
tools:context=".MainActivity" >

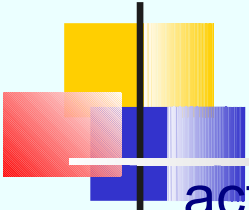
<ListView
android:id="@+id/listView1"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_centerHorizontal="true"
android:layout_centerVertical="true" >
</ListView>
</RelativeLayout>
```



Example (12)

activity_display_contacts.xml (1):

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/scrollView1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    tools:context=".DisplayContact" >
    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="370dp">
        <EditText
            android:id="@+id/editTextName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentLeft="true"
            android:layout_marginTop="5dp"
            android:layout_marginLeft="82dp"
            android:ems="10"
            android:inputType="text" >
        </EditText>
        <EditText
            android:id="@+id/editTextEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignLeft="@+id/editTextStreet"
            android:layout_below="@+id/editTextStreet"
            android:layout_marginTop="22dp"
            android:ems="10"
            android:inputType="textEmailAddress" />
        <TextView
            android:id="@+id/textView1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignBottom="@+id/editTextName"
            android:layout_alignParentLeft="true"
            android:text="@string/name"
            android:textAppearance="?android:attr/textAppearanceMedium" />
```



Example (13)

activity_display_contacts.xml (2):

```
<Button
android:id="@+id/button1"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignLeft="@+id/editTextCity"
android:layout_alignParentBottom="true"
android:layout_marginBottom="28dp"
android:onClick="run"
android:text="@string/save" />
<TextView
android:id="@+id/textView2"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignBottom="@+id/editTextEmail"
android:layout_alignLeft="@+id/textView1"
android:text="@string/email"
android:textAppearance="?android:attr/textAppearanceMedium" />
<TextView
android:id="@+id/textView5"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignBottom="@+id/editTextPhone"
android:layout_alignLeft="@+id/textView1"
android:text="@string/phone"
android:textAppearance="?android:attr/textAppearanceMedium" />
<TextView
android:id="@+id/textView4"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_above="@+id/editTextEmail"
android:layout_alignLeft="@+id/textView5"
android:text="@string/street"
android:textAppearance="?android:attr/textAppearanceMedium" />
```



Example (14)

activity_display_contacts.xml (3):

```
<EditText android:id="@+id/editTextCity"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignRight="@+id/editTextName"
android:layout_below="@+id/editTextEmail"
android:layout_marginTop="30dp"
android:ems="10"
android:inputType="text" />
<TextView android:id="@+id/textView3"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignBaseline="@+id/editTextCity"
android:layout_alignBottom="@+id/editTextCity"
android:layout_alignParentLeft="true"
android:layout_toLeftOf="@+id/editTextEmail"
android:text="@string/country"
android:textAppearance="?android:attr/textAppearanceMedium" />
<EditText android:id="@+id/editTextStreet"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignLeft="@+id/editTextName"
android:layout_below="@+id/editTextPhone"
android:ems="10"
android:inputType="text" >
<requestFocus />
</EditText>
<EditText android:id="@+id/editTextPhone"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignLeft="@+id/editTextStreet"
android:layout_below="@+id/editTextName"
android:ems="10"
android:inputType="phone|text" />
</RelativeLayout>
</ScrollView>
```



Example (15)

strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<string name="app_name">Address Book</string>
<string name="action_settings">Settings</string>
<string name="hello_world">Hello world!</string>
<string name="Add_New">Add New</string>
<string name="edit">Edit Contact</string>
<string name="delete">Delete Contact</string>
<string name="title_activity_display_contact">DisplayContact</string>
<string name="name">Name</string>
<string name="phone">Phone</string>
<string name="email">Email</string>
<string name="street">Street</string>
<string name="country">City/State/Zip</string>
<string name="save">Save Contact</string>
<string name="deleteContact">Are you sure, you want to delete
it.</string>
<string name="yes">Yes</string>
<string name="no">No</string>
</resources>
```



Example (16)

res/menu/mainmenu.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android" >
<item android:id="@+id/item1"
android:icon="@drawable/add"
android:title="@string/Add_New"
android:showAsAction="ifRoom|withText">
</item>
</menu>
```



Example (17)

res/menu/display_contact.xml:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android" >
  <item
    android:id="@+id/Edit_Contact"
    android:orderInCategory="100"
    android:title="@string/edit"/>
  <item
    android:id="@+id/Delete_Contact"
    android:orderInCategory="100"
    android:title="@string/delete"/>
</menu>
```



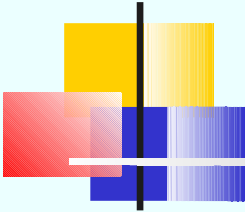
Example (18)

AndroidManifest.xml:

```
<application
    android:allowBackup="true"
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >

    <activity
        android:name="com.example.addressbook.MainActivity"
        android:label="@string/app_name" >
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" >
        </intent-filter>
    </activity>

    <activity
        android:name="com.example.addressbook.DisplayContact"
        android:label="@string/title_activity_display_contact" >
    </activity>
</application>
```



Example (19)

DisplayContact

Name

Phone

Street

Email

City/
State/Zip

DisplayContact

Name

Phone

Street

Email

City/
State/Zip

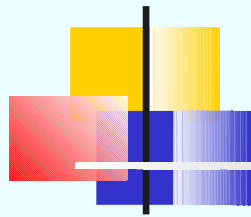
DisplayContact

Name

Phone

Are you sure

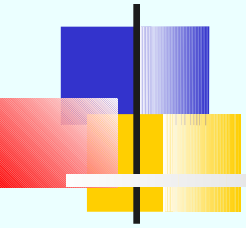
Are you sure, you want to delete it.



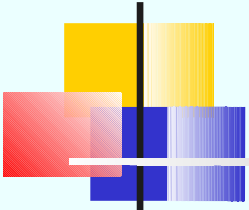
Exercises

App of student registration management:

- Management of students
- Management of subject and subject-class
- Management of student registration: a student registers in a subject-class of a subject. That will have a point for the student for the subject



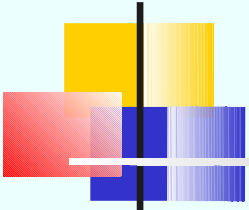
Fragment



Fragment

A Fragment = sub-activity.

- A fragment has its own layout and its own behavior with its own lifecycle
- can add or remove fragments in an activity while the activity is running.
- can combine multiple fragments in a single activity to build a multi-pane UI.
- A fragment can be used in multiple activities
- A fragment can implement a behavior that has no user interface component.



Create fragment

Step by step:

- decide how many fragments you want to use in an activity.
- create classes which will extend the *Fragment* class.
- create layout files in XML file.
- implement Content Provider queries to perform different database specific operations.
- modify activity file to define the actual logic of replacing fragments

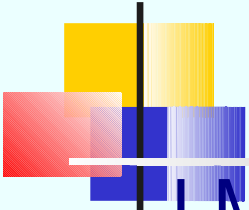


Example (1)

MainActivity.java:

```
package com.example.myfragments;
import android.os.Bundle;
import android.app.Activity;
import android.app.FragmentManager;
import android.app.FragmentTransaction;
import android.content.res.Configuration;
import android.view.WindowManager;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Configuration config = getResources().getConfiguration();
        FragmentManager fragmentManager = getFragmentManager();
        FragmentTransaction fragmentTransaction =
            fragmentManager.beginTransaction();
        if (config.orientation == Configuration.ORIENTATION_LANDSCAPE)
            LM_Fragment ls_fragment = new LM_Fragment();
            fragmentTransaction.replace(android.R.id.content, ls_fragment);
        }else{
            PM_Fragment pm_fragment = new PM_Fragment();
            fragmentTransaction.replace(android.R.id.content, pm_fragment);
        }
        fragmentTransaction.commit();
    }
}
```



Example (2)

LM_Fragment.java:

```
package com.example.myfragments;
import android.app.Fragment;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

public class LM_Fragment extends Fragment{
    @Override
    public View onCreateView(LayoutInflater inflater,
        ViewGroup container, Bundle savedInstanceState) {
        return inflater.inflate(R.layout.lm_fragment, container, false);
    }
}
```



Example (3)

PM_Fragment.java:

```
package com.example.myfragments;
import android.app.Fragment;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

public class PM_Fragment extends Fragment{
    @Override
    public View onCreateView(LayoutInflater inflater,
        ViewGroup container, Bundle savedInstanceState) {
        return inflater.inflate(R.layout.pm_fragment, container, false);
    }
}
```



Example (4)

lm_fragment.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="#7bae16">

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/landscape_message"
        android:textColor="#000000"
        android:textSize="20px" />

    <!-- More GUI components go here -->

</LinearLayout>
```



Example (5)

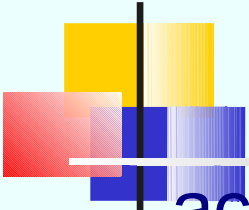
pm_fragment.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="#7bae16">

    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/portrait_message"
        android:textColor="#000000"
        android:textSize="20px" />

    <!-- More GUI components go here -->

</LinearLayout>
```



Example (6)

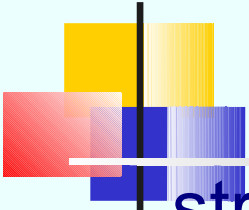
activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="horizontal">

    <fragment
        android:name="com.example.fragments"
        android:id="@+id/lm_fragment"
        android:layout_weight="1"
        android:layout_width="0dp"
        android:layout_height="match_parent" />

    <fragment
        android:name="com.example.fragments"
        android:id="@+id/pm_fragment"
        android:layout_weight="2"
        android:layout_width="0dp"
        android:layout_height="match_parent" />

</LinearLayout>
```

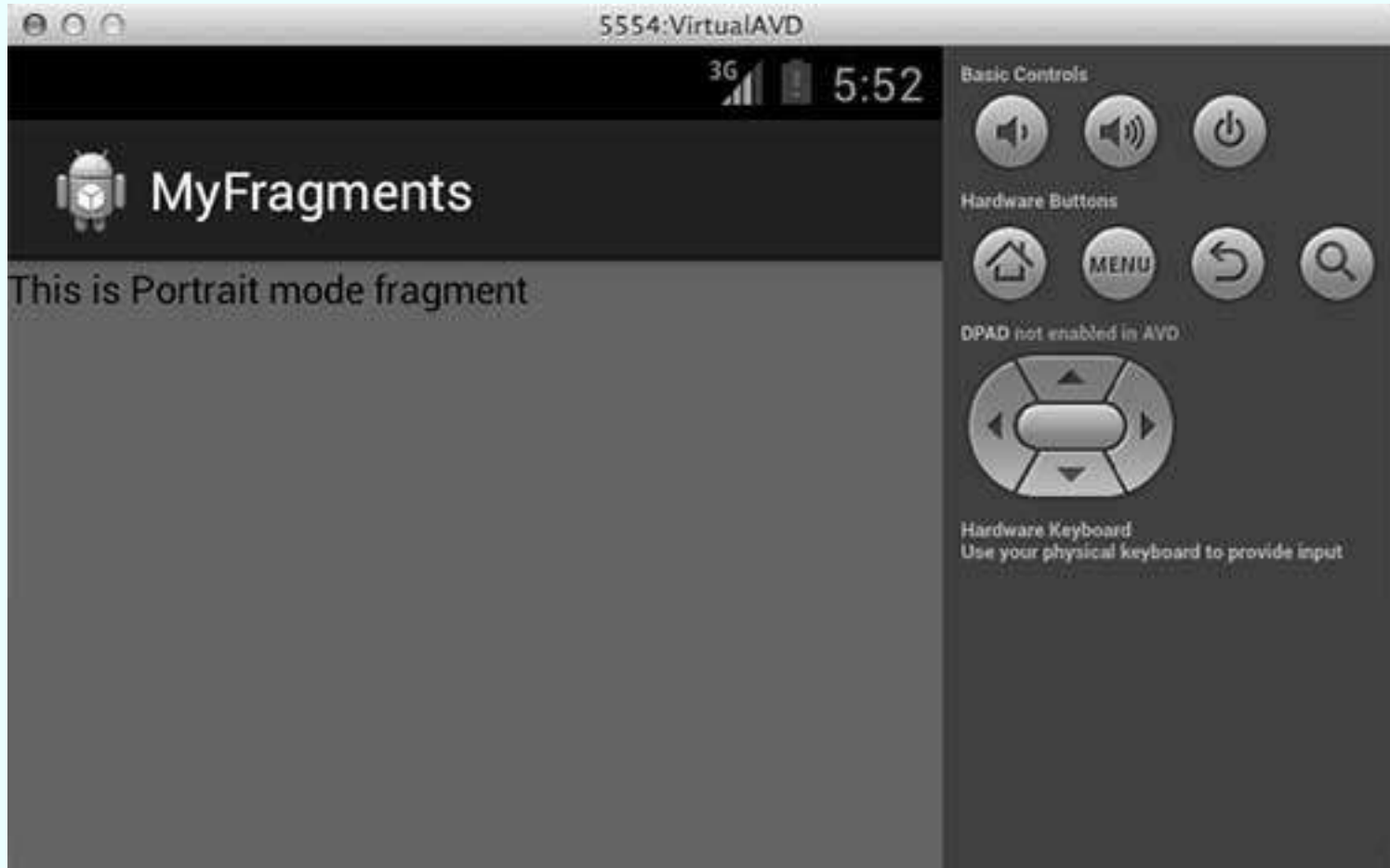


Example (7)

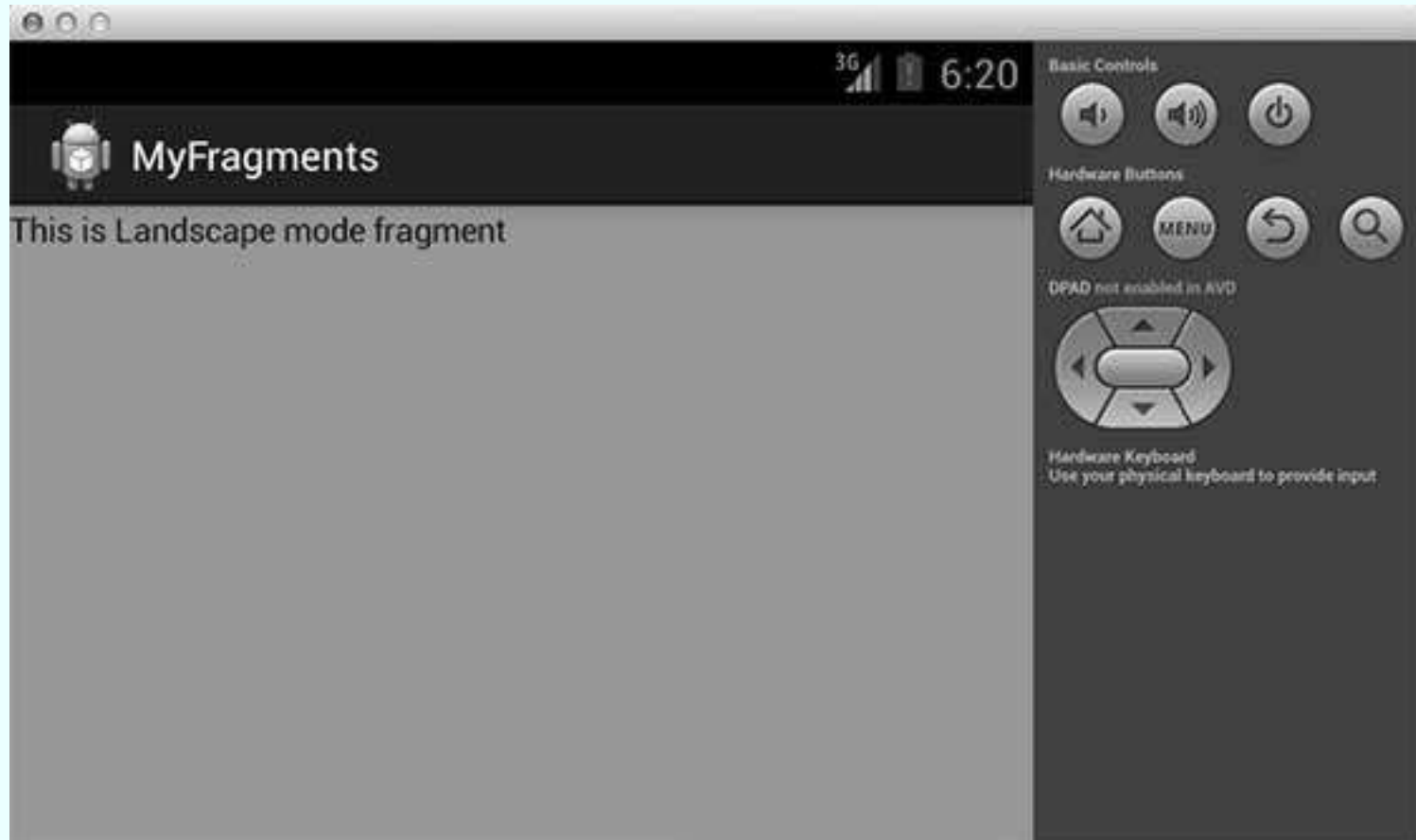
strings.xml:

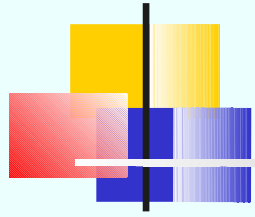
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">MyFragments</string>
    <string name="action_settings">Settings</string>
    <string name="hello_world">Hello world!</string>
    <string name="landscape_message">This is Landscape mode fragment
</string>
    <string name="portrait_message">This is Portrait mode fragment
</string>
</resources>
```

Example (8)



Example (9)

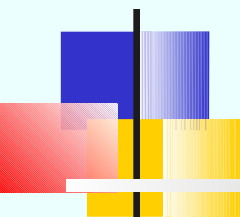




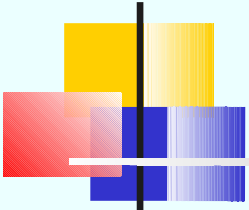
Exercises

With the student management example:

- Create an activity of management
- Create three fragment: add, search+edit, search+delete



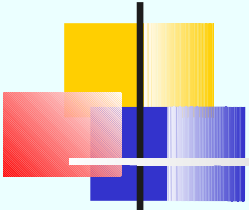
Intents & filters



Intent

Intent = object to carry data from one component to other when calling:

- `Context.startActivity()`
- `Context.startService()`
- `Context.sendBroadcast()`

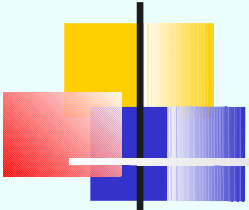


Implicit Intent

Using to call other application:

```
// Implicit Intent by specifying a URI
Intent i = new Intent(Intent.ACTION_VIEW,
Uri.parse("http://www.example.com"));

// Starts Implicit Activity
startActivity(i);
```



Explicit Intent

In the calling activity:

```
// Explicit Intent by specifying its class name
Intent i = new Intent(this, TargetActivity.class);
i.putExtra("Key1", "ABC");
i.putExtra("Key2", "123");
// Starts TargetActivity
startActivity(i);
```

In the called activity:

```
// Get bundle object at appropriate place in your code
Bundle extras = getIntent().getExtras();
// Extract data using passed keys
String value1 = extras.getString("Key1");
String value2 = extras.getString("Key2");
```



Example (1)

MainActivity.java (1):

```
package com.example.intentdemo;
import android.net.Uri;
import android.os.Bundle;
import android.app.Activity;
import android.content.Intent;
import android.view.Menu;
import android.view.View;
import android.widget.Button;

public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button startBrowser = (Button) findViewById(R.id.start_browser);
        startBrowser.setOnClickListener(new View.OnClickListener() {
            public void onClick(View view) {
                Intent i = new Intent(android.content.Intent.ACTION_VIEW,
                    Uri.parse("http://www.example.com"));
                startActivity(i);
            }
        });
    }
}
```

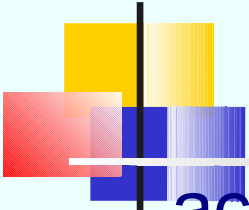


Example (2)

MainActivity.java (2):

```
        Button startPhone = (Button) findViewById(R.id.start_phone);
        startPhone.setOnClickListener(new View.OnClickListener() {
            public void onClick(View view) {
                Intent i = new Intent(android.content.Intent.ACTION_VIEW,
                    Uri.parse("tel:9510300000"));
                startActivity(i);
            }
        });
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action
        // bar if it is present.
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```



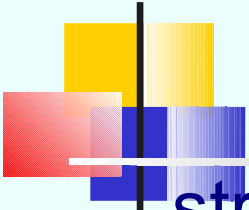
Example (3)

activity_main.xml:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <Button android:id="@+id/start_browser"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/start_browser"/>

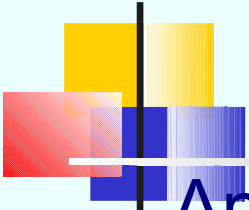
    <Button android:id="@+id/start_phone"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/start_phone" />
</LinearLayout>
```



Example (4)

strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">IntentDemo</string>
    <string name="action_settings">Settings</string>
    <string name="hello_world">Hello world!</string>
    <string name="start_browser">Start Browser</string>
    <string name="start_phone">Start Phone</string>
</resources>
```



Example (5)

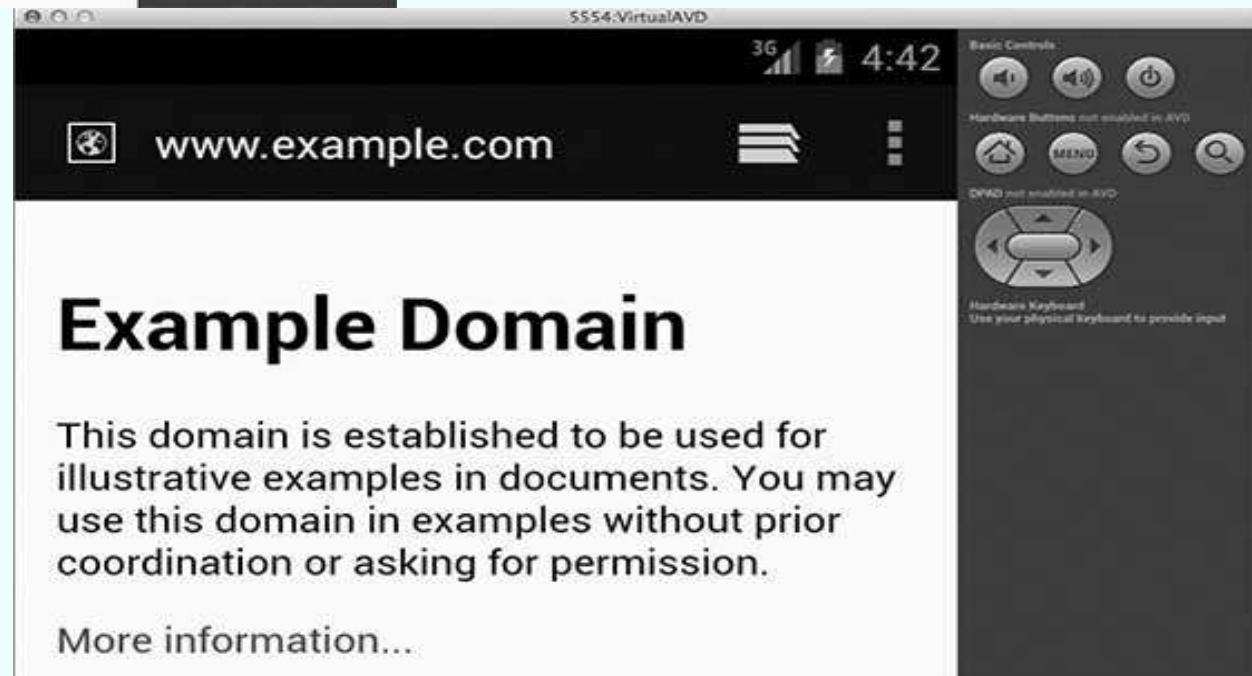
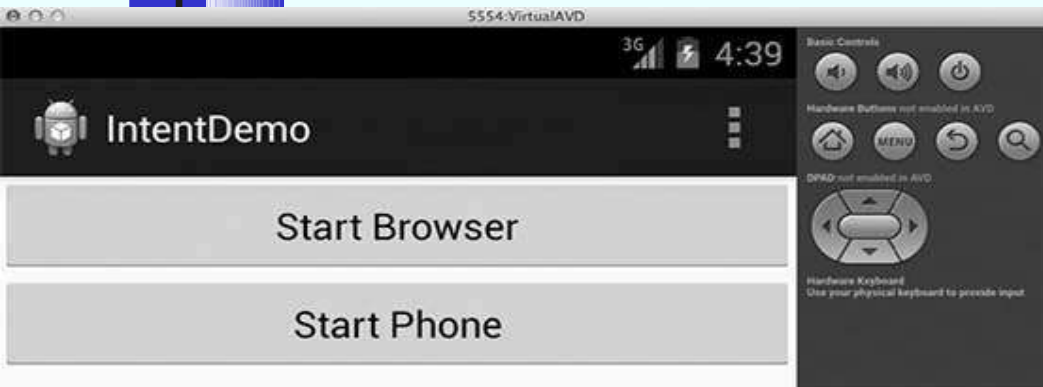
AndroidManifest.xml:

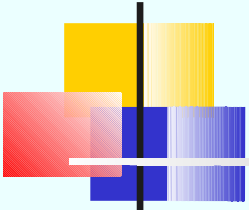
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.intentdemo"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="17" />

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name="com.example.intentdemo.MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Example (6)



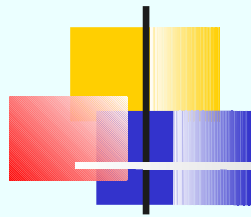


Filter

Used in Manifest to specify an activity can be invoked with some kinds of action, category and data:

```
<activity android:name=".CustomActivity"
    android:label="@string/app_name">

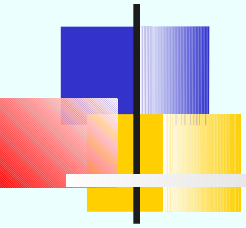
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <action android:name="com.example.intentdemo.LAUNCH" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:scheme="http" />
    </intent-filter>
</activity>
```



Exercises

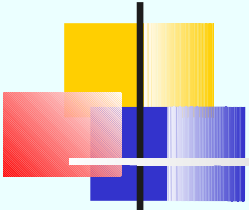
With the student management example:

- Create an activity of search and display the result
- Create an other activity to edit a student when click a student from the result list (the first activity have to send the chosen student to the sencond activity to edit)



UI

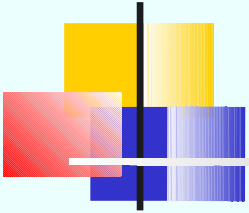




Layouts

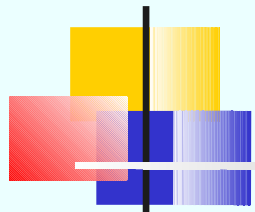
Layout = the way to organize view components

- LinearLayout
- RelativeLayout
- TableLayout
- AbsoluteLayout
- FrameLayout
- ListView
- GridView



Layout attributes

- id
- layout_width
- layout_height
- layout_margin[Top, Bottom, Left, Right]
- layout_gravity
- layout_weight
- layout_x
- layout_y
- layout_padding[Top, Bottom, Left, Right]



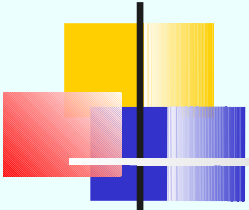
View id

In the .xml:

```
android:id="@+id/my_button"
```

In the code .java:

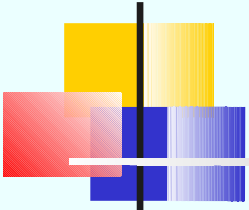
```
Button myButton = (Button) findViewById(R.id.my_button);
```



UI controls

UI controls list:

- TextView, EditText, AutoCompleteTextView
- Button, ImageButton
- CheckBox
- ToggleButton
- RadioButton, RadioGroup
- ProgressBar
- Spinner
- TimePicker
- DatePicker



Create UI controls

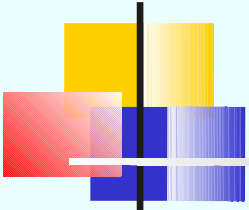
In the .xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView android:id="@+id/text_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="I am a TextView" />
</LinearLayout>
```

In the code .java:

```
TextView txt = (TextView) findViewById(R.id.text_id);
```



Events handling

UI controls event list:

- `onClick()`
- `onLongClick()`
- `onFocusChange()`
- `onkey()`
- `onTouch()`
- `onMenuItemClick()`



Event handling with inner class (1)

MainActivity.java (1):

```
package com.example.eventdemo;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends Activity {

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```



Event handling with inner class (2)

MainActivity.java (2):

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //--- find both the buttons---
    Button sButton = (Button) findViewById(R.id.button_s);
    Button lButton = (Button) findViewById(R.id.button_l);
    // -- register click event with first button ---
    sButton.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            // --- find the text view --
            TextView txtView = (TextView) findViewById(R.id.text_i
            // -- change text size --
            txtView.setTextSize(14);
        }
    });
    // -- register click event with second button ---
    lButton.setOnClickListener(new View.OnClickListener() {
        public void onClick(View v) {
            // --- find the text view --
            TextView txtView = (TextView) findViewById(R.id.text_i
            // -- change text size --
            txtView.setTextSize(24);
        }
    });
}
```



Event handling with inner class (3)

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <Button
        android:id="@+id/button_s"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:text="@string/button_small"/>

    <Button
        android:id="@+id/button_l"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:text="@string/button_large"/>

    <TextView
        android:id="@+id/text_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:capitalize="characters"
        android:text="@string/hello_world" />
</LinearLayout>
```



Event handling with inner class (4)

strings.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">EventDemo</string>
    <string name="action_settings">Settings</string>
    <string name="hello_world">Hello world!</string>
    <string name="button_small">Small Font</string>
    <string name="button_large">Large Font</string>
</resources>
```



Event handling with inner class (5)

AndroidManifest.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.guidemo"
    android:versionCode="1"
    android:versionName="1.0" >

    <uses-sdk
        android:minSdkVersion="8"
        android:targetSdkVersion="17" />

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >

        <activity
            android:name="com.example.guidemo.MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Event handling with inner class (6)





Event handling with listener (1)

MainActivity.java (1):

```
package com.example.eventdemo;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends Activity implements OnClickListener {

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```



Event handling with inner class (2)

MainActivity.java (2):

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //--- find both the buttons---
    Button sButton = (Button) findViewById(R.id.button_s);
    Button lButton = (Button) findViewById(R.id.button_l);
    // -- register click event with first button ---
    sButton.setOnClickListener(this);

    // -- register click event with second button ---
    lButton.setOnClickListener(this);
}
```



Event handling with inner class (3)

MainActivity.java (3):

```
//--- Implement the OnClickListener callback
public void onClick(View v) {
    if(v.getId() == R.id.button_s){
        // --- find the text view --
        TextView txtView = (TextView) findViewById(R.id.text_id);
        // -- change text size --
        txtView.setTextSize(14);
        return;
    }
    if(v.getId() == R.id.button_l){
        // --- find the text view --
        TextView txtView = (TextView) findViewById(R.id.text_id);
        // -- change text size --
        txtView.setTextSize(24);
        return;
    }
}
```



Event handling in xml (1)

MainActivity.java (1):

```
package com.example.eventdemo;
import android.os.Bundle;
import android.app.Activity;
import android.view.Menu;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends Activity{

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main, menu);
        return true;
    }
}
```



Event handling in xml (2)

MainActivity.java (2):

```
//--- Implement the event handler for the first button.
public void doSmall(View v) {
    // --- find the text view --
    TextView txtView = (TextView) findViewById(R.id.text_id);
    // -- change text size --
    txtView.setTextSize(14);
    return;
}

//--- Implement the event handler for the second button.
public void doLarge(View v) {
    // --- find the text view --
    TextView txtView = (TextView) findViewById(R.id.text_id);
    // -- change text size --
    txtView.setTextSize(24);
    return;
}
```



Event handling in xml (3)

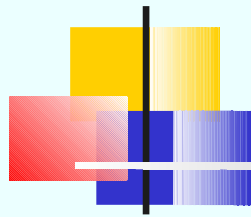
activity-main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <Button
        android:id="@+id/button_s"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:text="@string/button_small"
        android:onClick="doSmall"/>

    <Button
        android:id="@+id/button_l"
        android:layout_height="wrap_content"
        android:layout_width="match_parent"
        android:text="@string/button_large"
        android:onClick="doLarge"/>

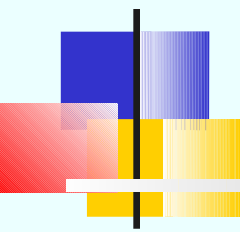
    <TextView
        android:id="@+id/text_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:capitalize="characters"
        android:text="@string/hello_world" />
</LinearLayout>
```



Exercises

With the student management example:

- Create an activity of search and display the result: display the result when typing the text in the keyword



Styles & themes



UI without style

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView
        android:id="@+id/text_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:capitalize="characters"
        android:textColor="#00FF00"
        android:typeface="monospace"
        android:text="@string/hello_world" />
</LinearLayout>
```



define style

res/values/style.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="CustomFontStyle">
        <item name="android:layout_width">fill_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:capitalize">characters</item>
        <item name="android:typeface">monospace</item>
        <item name="android:textSize">12pt</item>
        <item name="android:textColor">#00FF00</item>/>
    </style>
</resources>
```



Using style in UI xml

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView
        android:id="@+id/text_id"
        style="@style/CustomFontStyle"
        android:text="@string/hello_world" />
</LinearLayout>
```



Style inheritance (1)

style.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="CustomFontStyle.LargeFont">
        <item name="android:textSize">20ps</item>
    </style>
</resources>
```

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="CustomFontStyle.LargeFont.Red">
        <item name="android:textColor">#FF0000</item>/>
    </style>
</resources>
```



Style inheritance (2)

style.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="CustomFontStyle"
        parent="@android:style/TextAppearance">

        <item name="android:layout_width">fill_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:capitalize">characters</item>
        <item name="android:typeface">monospace</item>
        <item name="android:textSize">12pt</item>
        <item name="android:textColor">#00FF00</item>/>
    </style>
</resources>
```



Theme

Used in manifest.xml

```
<application android:theme="@style/CustomFontStyle">
```

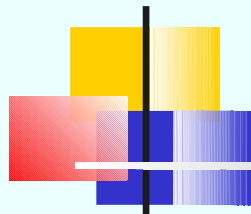
```
<activity android:theme="@style/CustomFontStyle">
```

defined in style.xml:

```
<style name="CustomTheme" parent="android:Theme.Light">
```

```
...
```

```
</style>
```

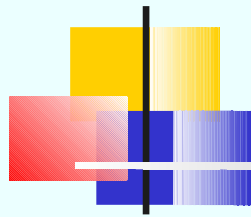


Exercises (1)

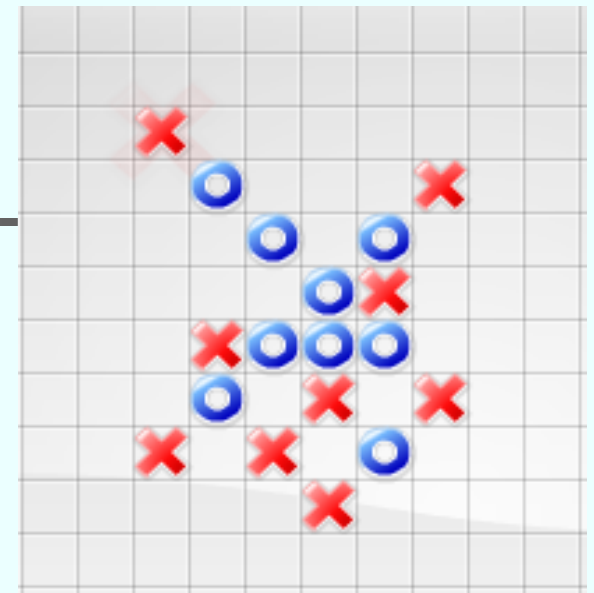
5	1					2		
				1	8	9		3
		3	9				1	
		7		9			5	
	3		8	4				2
9	5			6			3	
		8			9	6	7	
6	2		7					
		1			6	8		5

Game of sudoku:

- Generate a sudoku and display to play
- Enable to click on empty cells
- Enable to insert a number into an empty cell
- Verify if a filled cell is valid
- Verify if the sudoku is solved
- Count the time for each win match
- Save the best time ever for each player

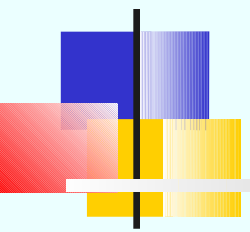


Exercises (2)



Game of gomoku (caro):

- Generate an empty gomoku table and display to play
- Enable to click on empty cells for player
- Able to play againsts player
- Verify if a player won
- Count the time for each win match
- Save the statistical time and win/lost for each player



Questions?
